# Laboratory Practice III

Complete Theory, Algorithms, Python Programs, Complexity Analysis, and Viva Preparation

DAA Experiments 1–4 and Machine Learning Experiments 1–5

Study Notes Based on the Laboratory Experiment List and Our Tutoring Discussion

Academic Year 2024–2025 syllabus reference
Prepared as a practical-examination study guide

**Abstract**

These notes consolidate the theory, algorithms, Python implementations, line-by-line explanations, complexity analysis, evaluation measures, common mistakes, and viva questions for Design and Analysis of Algorithms (DAA) Experiments 1–4 and Machine Learning (ML) Experiments 1–5. The covered DAA topics are recursive and iterative Fibonacci numbers, Huffman encoding, fractional knapsack, and 0–1 knapsack. The covered ML topics are Uber fare prediction using regression, email spam classification using K-Nearest Neighbors (KNN) and Support Vector Machine (SVM), and bank-customer churn prediction using an Artificial Neural Network (ANN). The ML coverage includes Uber fare regression, email spam detection, bank-customer churn prediction, diabetes classification using KNN, and sales clustering using K-Means or hierarchical clustering. The emphasis is on understanding why each step is performed, not memorizing code without comprehension.

**How to use this guide.** Read the problem statement and intuition first. Trace the algorithm on a small input. Then study the code in blocks and explain every important line aloud. Finally, answer the viva questions without looking at the answers. Code may require small adjustments for the exact CSV filename, column names, package versions, or dataset variant supplied by the instructor.

## Contents

**1 Scope, Practical Workflow, and Coding Conventions** **6**

1.1 What a practical answer should contain . . . . . 6

1.2 Basic Python conventions . . . . . 6

1.3 How to reason about complexity . . . . . 6

**Part I: Design and Analysis of Algorithms** **8**

**2 DAA Experiment 1: Fibonacci Numbers Using Iteration and Recursion 8**
2.1 Aim and problem definition . . . 8
2.2 What is asked and simple coding flow . . . 8
2.3 Approach A: recursive solution . . . 8
2.4 Deeper theory: recursion, recurrence relations, and call stacks . . . 8
2.5 Why naive recursion is slow . . . 9
2.6 Approach B: iterative / non-recursive solution . . . 10
2.7 Complexity comparison . . . 10
2.8 Dry run . . . 10
2.9 Common errors . . . 11
2.10 Viva questions . . . 11

**3 DAA Experiment 2: Huffman Encoding Using a Greedy Strategy 13**
3.1 Aim and intuition . . . 13
3.2 What is asked and simple coding flow . . . 13
3.3 Greedy choice . . . 13
3.4 Why prefix-free codes and the greedy rule matter . . . 13
3.5 Python implementation . . . 14
3.6 Code walkthrough . . . 15
3.7 Encoding and decoding . . . 16
3.8 Complexity . . . 16
3.9 Advantages, limitations, and viva . . . 16

**4 DAA Experiment 3: Fractional Knapsack Using a Greedy Method 18**
4.1 Problem statement . . . 18
4.2 What is asked and simple coding flow . . . 18
4.3 Algorithm . . . 18
4.4 Python implementation . . . 18
4.5 Walkthrough and dry run . . . 19
4.6 Why the greedy rule works here . . . 19
4.7 Proof intuition and the limit of the greedy strategy . . . 19
4.8 Complexity and comparison with 0–1 knapsack . . . 20
4.9 Viva questions . . . 20

**5 DAA Experiment 4: 0–1 Knapsack Using Dynamic Programming 22**
5.1 Problem definition . . . 22
5.2 What is asked and simple coding flow . . . 22

5.3 Dynamic programming idea . . . 22
5.4 Dynamic programming in detail: state, transition, and optimal substructure 22
5.5 Python implementation . . . 23
5.6 Understanding the code . . . 24
5.7 Complexity . . . 24
5.8 Fractional versus 0–1 knapsack . . . 24
5.9 Viva questions . . . 24

**6 Machine Learning Foundations Used in These Experiments 26**
6.1 Supervised learning . . . 26
6.2 A more complete conceptual model of machine learning . . . 26
6.3 Typical ML pipeline . . . 27
6.4 Common preprocessing operations . . . 27
6.5 Reproducibility . . . 28

**7 ML Experiment 1: Uber Fare Prediction with Linear Regression and Random Forest 29**
7.1 Aim and dataset . . . 29
7.2 What is asked and simple coding flow . . . 29
7.3 Linear regression theory . . . 29
7.4 Linear regression: assumptions, fitting, and interpretation . . . 29
7.5 Random forest regression theory . . . 30
7.6 Random forests: how tree ensembles reduce variance . . . 30
7.7 Outlier identification and correlation . . . 31
7.8 Complete example code . . . 31
7.9 Code explanation . . . 32
7.10 Evaluation measures . . . 33
7.11 Practical caveats . . . 33
7.12 Viva questions . . . 34

**8 ML Experiment 2: Email Spam Detection Using KNN and SVM 35**
8.1 Aim and dataset . . . 35
8.2 What is asked and simple coding flow . . . 35
8.3 K-Nearest Neighbors . . . 35
8.4 KNN in depth: distance, scaling, and the choice of $K$ . . . 35
8.5 Support Vector Machine . . . 36
8.6 SVM in depth: margins, support vectors, and kernels . . . 36
8.7 Complete example code . . . 37

8.8 Explanation of the pipeline . . . 38
8.9 Confusion matrix and classification metrics . . . 38
8.10 Comparing KNN and SVM . . . 39
8.11 Viva questions . . . 39

**9 ML Experiment 3: Bank Customer Churn Prediction Using an ANN 41**
9.1 Aim and dataset . . . 41
9.2 What is asked and simple coding flow . . . 41
9.3 Artificial Neural Network theory . . . 41
9.4 Artificial neural networks: units, layers, and learned parameters . . . 41
9.5 Forward propagation, loss, and backpropagation . . . 42
9.6 Training mathematics: forward pass, loss, gradients, and optimization . . . 42
9.7 Complete example code . . . 43
9.8 Preprocessing explained . . . 45
9.9 Understanding the network definition . . . 45
9.10 Compile, fit, predict, and evaluate . . . 46
9.11 Churn-specific interpretation and improvements . . . 46
9.12 Viva questions . . . 47

**10 Cross-Experiment Comparison and Exam Recall Tables 48**
10.1 DAA strategies . . . 48
10.2 ML methods . . . 48
10.3 Functions that are easy to confuse . . . 48

**11 Common Errors, Debugging, and Responsible Reporting 50**
11.1 Dataset and column errors . . . 50
11.2 Preprocessing and leakage . . . 50
11.3 Metric mistakes . . . 50
11.4 Reproducible reporting checklist . . . 50

**12 Theory Answer-Writing Framework and Extended Revision 52**
12.1 How to write a strong DAA long answer . . . 52
12.2 How to write a strong ML long answer . . . 52
12.3 Metric interpretation: what a score can and cannot say . . . 52
12.4 Worked metric example . . . 53
12.5 Worked knapsack distinction . . . 53
12.6 Practical checklist before submitting code . . . 53
12.7 Dataset assumptions and reproducibility notes . . . 53

12.8 Additional viva questions for understanding . . . . . 54

**13 Final Viva Practice: Questions with Short Answers 54**

13.1 DAA revision . . . . . 54
13.2 Machine-learning revision . . . . . 55
13.3 Mini oral-exam routine . . . . . 55

**14 Machine Learning Experiment 4: Diabetes Prediction Using K-Nearest Neighbors 56**

14.1 Experiment statement and objectives . . . . . 56
14.2 What is asked and simple coding flow . . . . . 56
14.3 What KNN does . . . . . 56
14.4 Why feature scaling is essential . . . . . 56
14.5 Recommended experimental workflow . . . . . 57
14.6 Illustrative Python implementation . . . . . 57
14.7 Understanding the confusion matrix . . . . . 58
14.8 Metrics and their interpretation . . . . . 59
14.9 Choosing $K$, advantages, and limitations . . . . . 59
14.10Common viva questions . . . . . 59

**15 Machine Learning Experiment 5: Customer Segmentation Using Clustering 60**

15.1 Experiment statement and objectives . . . . . 60
15.2 What is asked and simple coding flow . . . . . 60
15.3 Supervised learning versus clustering . . . . . 61
15.4 K-Means algorithm . . . . . 61
15.5 Feature selection and preparation . . . . . 61
15.6 The elbow method . . . . . 61
15.7 Illustrative K-Means implementation . . . . . 62
15.8 How to interpret clusters . . . . . 63
15.9 Hierarchical clustering alternative . . . . . 63
15.10Common viva questions . . . . . 63

**16 Final Exam Checklist 64**

# 1 Scope, Practical Workflow, and Coding Conventions

## 1.1 What a practical answer should contain

For an algorithm experiment, a complete practical write-up usually includes:

1. **Aim:** the task to be solved.
2. **Theory:** the central idea and why the chosen strategy works.
3. **Algorithm:** ordered steps or pseudocode.
4. **Program:** readable, executable code.
5. **Sample input/output:** a small example that can be checked manually.
6. **Complexity:** time and auxiliary-space analysis, including assumptions.
7. **Conclusion:** what the program demonstrates.

For a machine-learning experiment, add dataset description, target/features, preprocessing, train/test split, model training, metrics, and a brief interpretation. Never report an accuracy or R-squared value as if it were guaranteed: actual results depend on the dataset, preprocessing, random split, software versions, and model settings.

## 1.2 Basic Python conventions

Python uses indentation to define code blocks. A colon introduces a block, such as after `if`, `for`, `while`, `def`, or `class`. A variable stores a value. A function groups reusable instructions and can return a result. A list stores an ordered collection; a dictionary maps keys to values; a tuple is an immutable sequence. Array indices normally start at zero.

For example:

```
def square(number):
    result = number * number
    return result

print(square(5))  # 25
```

The function receives an argument, computes a value, and returns it. The `return` statement sends the value back to the caller. A `print` statement displays a value but does not replace a return value.

## 1.3 How to reason about complexity

Time complexity describes how the number of operations grows with input size. Space complexity describes memory usage. Big-O notation gives an asymptotic upper bound, commonly used to express worst-case growth. It is not a direct measurement in seconds.

| Complexity | Typical interpretation |
|---|---|
| $O(1)$ | Constant work |
| $O(\log n)$ | Logarithmic growth |
| $O(n)$ | Linear growth |
| $O(n \log n)$ | Typical efficient comparison sorting scale |
| $O(n^2)$ | Quadratic growth, often nested loops |
| $O(2^n)$ | Exponential growth |
| $O(n!)$ | Factorial growth |

When analyzing a program, identify the input-size variable, count loop iterations or recursive calls, and distinguish auxiliary memory from memory required to store the input or output.

# Part I: Design and Analysis of Algorithms

## 2 DAA Experiment 1: Fibonacci Numbers Using Iteration and Recursion

### 2.1 Aim and problem definition

Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze their time and space complexity. The Fibonacci sequence is defined by

$$F(0) = 0, \qquad F(1) = 1, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

The first values are $0, 1, 1, 2, 3, 5, 8, 13, \ldots$. Each term after the first two is the sum of the preceding two terms. This guide uses the convention that the input $n$ asks for $F(n)$, so the input 6 produces 8.

### 2.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Input n -> validate n -> choose recursive / iterative method -> calculate
        F(n) -> print result -> compare time and space complexity
```

### 2.3 Approach A: recursive solution

### 2.4 Deeper theory: recursion, recurrence relations, and call stacks

Recursion is a problem-solving technique in which a function solves a problem by calling itself on smaller instances of the same problem. A valid recursive design needs three parts: (1) a base case that can be answered directly, (2) a recursive case that reduces the problem, and (3) progress toward a base case. Fibonacci satisfies this structure because an input $n \geq 2$ is reduced to $n-1$ and $n-2$. If the input is not reduced, recursion may never terminate.

A recursive call creates an activation record on the program's call stack. The record conceptually stores the current arguments, local state, and where execution must continue after the called function returns. When a base case is reached, the result returns to the previous call; calls then unwind in reverse order. This explains why recursive code can look short while using substantial stack memory.

For example, evaluating $F(4)$ produces the call structure $F(4) = F(3) + F(2)$, $F(3) = F(2) + F(1)$, and each $F(2) = F(1) + F(0)$. The same subproblem $F(2)$ is solved more than once. The inefficiency is not caused by addition itself; it comes from recomputing identical subproblems. This is the key observation that motivates memoization and dynamic programming.

The recurrence for the naive running time is $T(n) = T(n-1) + T(n-2) + O(1)$. Since the call tree branches and its depth grows with $n$, the total number of calls grows exponentially. A common safe upper bound is $O(2^n)$. The stack depth, however, is only linear because a path descends by at least one at every call, giving $O(n)$ auxiliary stack space. Do not confuse the number of calls with the maximum number of calls active simultaneously.

**When recursion is useful.** Recursion often mirrors mathematical definitions and is natural for tree traversal, divide-and-conquer, backtracking, and nested structures. Its costs include function-call overhead, stack use, and possible recursion-depth limits. The best approach depends on the problem, not on the belief that recursion is always better or worse than iteration.

**Memoization extension.** Memoization stores the result of each input $n$ after the first calculation. Subsequent requests reuse the stored result. Since only $F(0), F(1), \ldots, F(n)$ are calculated once, time becomes $O(n)$ and storage is $O(n)$. Iteration achieves $O(n)$ time with $O(1)$ auxiliary space when only the final value is needed. The recursive definition directly mirrors the mathematical recurrence. If $n$ is zero or one, return the base value. Otherwise, recursively compute the two preceding Fibonacci numbers and add them.

```python
def fibonacci_recursive(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

n = int(input("Enter n: "))
print("Fibonacci number:", fibonacci_recursive(n))
```

**Line-by-line reasoning.**

- `def fibonacci_recursive(n)` defines a function with input `n`.
- The negative-input check prevents a value outside the stated domain.
- The two base cases stop recursion. Without them, the calls would continue until an error.
- The final line follows the recurrence and combines two recursive results.
- The main program reads an integer and prints the returned value.

## 2.5 Why naive recursion is slow

For $n \geq 2$, the function calls itself twice. The call tree repeats work: for example, computing $F(5)$ computes $F(3)$ more than once. The recurrence for the running time is approximately

$$T(n) = T(n-1) + T(n-2) + O(1).$$

Its growth is exponential, commonly bounded as $O(2^n)$ for a simple upper-bound analysis. The maximum recursion depth is $O(n)$, so auxiliary call-stack space is $O(n)$. This is a good demonstration of recursion, but it is not efficient for large $n$.

## 2.6 Approach B: iterative / non-recursive solution

The iterative method stores only the previous two values. It avoids repeated recursive calls and does not use a growing recursive call stack.

```python
def fibonacci_iterative(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 0

    previous, current = 0, 1
    for _ in range(2, n + 1):
        previous, current = current, previous + current
    return current

n = int(input("Enter n: "))
print("Fibonacci number:", fibonacci_iterative(n))
```

The pair `previous, current` initially represents $F(0)$ and $F(1)$. Each iteration moves the pair forward: the old current becomes the previous value, and the new current becomes the sum. For $n = 1$, the loop executes zero times and the function correctly returns one. For $n = 2$, it executes once and returns one.

## 2.7 Complexity comparison

| Method | Time complexity | Auxiliary space |
|---|---|---|
| Naive recursion | $O(2^n)$ upper bound | $O(n)$ call stack |
| Iteration | $O(n)$ | $O(1)$ |

The iterative algorithm is preferred when the goal is to calculate one Fibonacci number efficiently. The recursive version is valuable for demonstrating how a mathematical recurrence maps to a recursive function. Memoization can improve the recursive version to $O(n)$ time by caching previously calculated values, at the cost of $O(n)$ memory.

## 2.8 Dry run

For $n = 5$, the iterative state progresses as follows:

| Iteration | `previous` before update | `current` before update | New pair |
|---|---|---|---|
| Initial | – | – | $(0, 1)$ |
| 1 | 0 | 1 | $(1, 1)$ |
| 2 | 1 | 1 | $(1, 2)$ |
| 3 | 1 | 2 | $(2, 3)$ |
| 4 | 2 | 3 | $(3, 5)$ |

The loop runs from 2 through $n$ inclusive. Therefore, for $n = 5$, four updates are performed and the returned value is 5.

## 2.9 Common errors

- Confusing the sequence position with the value: under this convention, $F(6) = 8$.
- Omitting the base cases, causing unbounded recursion.
- Using a loop range that stops too early, such as `range(2, n)`.
- Claiming the naive recursive method is $O(n)$; its repeated calls make it exponential.
- Forgetting that Python integers can grow beyond fixed-width integer limits, although arithmetic on very large integers becomes more expensive.

## 2.10 Viva questions

1. **What is recursion?** A function solving a problem by calling itself on smaller instances.
2. **What is a base case?** A condition that terminates recursive calls with a direct result.
3. **Why is naive Fibonacci recursion inefficient?** It recalculates the same subproblems many times.
4. **What is the iterative time complexity?** $O(n)$, because the loop performs a number of updates proportional to $n$.
5. **What is memoization?** Storing results of solved subproblems to avoid repeated work.
6. **What are the base values?** $F_0 = 0$ and $F_1 = 1$.
7. **What is the recurrence relation?** $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.
8. **What does the input $n$ represent?** Here it represents the index of the Fibonacci number, starting at index 0.
9. **What is the time complexity of naive recursion?** Exponential, commonly bounded as $O(2^n)$.
10. **What is the auxiliary space of naive recursion?** $O(n)$ call-stack depth.
11. **What is the iterative time complexity?** $O(n)$.
12. **What is the iterative space complexity when only two previous values are stored?** $O(1)$ auxiliary space.
13. **What is overlapping subproblem work?** The same Fibonacci values are recalculated in many recursive branches.
14. **How does memoization help?** It caches solved values so each subproblem is calculated once.
15. **What happens for $n = 0$?** The result is 0 under the zero-indexed convention.
16. **Why validate negative input?** Negative indices are outside the usual practical definition used by this program.
17. **What is a stack frame?** The execution state stored for an active function call.

18. **Why can recursion cause an error for large n?** It may exceed the interpreter recursion limit and performs many repeated calls.
19. **What is an algorithm recurrence?** An equation describing the cost or value in terms of smaller input instances.
20. **What is the first few sequence values?** 0, 1, 1, 2, 3, 5, 8, 13, ...

# 3 DAA Experiment 2: Huffman Encoding Using a Greedy Strategy

## 3.1 Aim and intuition

Implement Huffman encoding using a greedy strategy. Huffman coding is a lossless compression method that assigns shorter binary codes to more frequent symbols and longer codes to less frequent symbols. It creates a prefix-free code: no symbol's code is the prefix of another symbol's code. This property allows an encoded bitstream to be decoded unambiguously without separators between symbols.

## 3.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Input symbols and frequencies -> build min-heap -> remove two
minimum-frequency nodes -> merge them -> repeat until one tree remains ->
assign 0/1 codes -> encode text
```

## 3.3 Greedy choice

## 3.4 Why prefix-free codes and the greedy rule matter

Huffman coding is a lossless compression method: it represents frequent symbols with shorter bit strings and infrequent symbols with longer strings, without losing the ability to recover the original symbol sequence. A fixed-width code for $m$ symbols needs roughly $\lceil \log_2 m \rceil$ bits per symbol. A variable-length code can use fewer bits on average when symbol frequencies are uneven.

A central requirement is that the code be *prefix-free*: no complete codeword is the prefix of another complete codeword. For example, if one symbol were coded as `0` and another as `01`, the bit stream `01` would be ambiguous. In a prefix-free code, a decoder can read bits from left to right and identify a symbol as soon as a leaf of the code tree is reached.

The Huffman construction repeatedly selects the two nodes with the lowest frequencies and merges them into a parent whose frequency is their sum. The selected nodes become siblings. This is a greedy algorithm because it makes the locally smallest-frequency merge at each step. The exchange-property intuition is that in an optimal binary prefix-code tree, two least-frequent symbols can be placed as deepest sibling leaves; merging them reduces the problem to a smaller instance with their combined frequency. Repeating this argument motivates the greedy choice.

The frequency of a symbol is a weight, not necessarily a probability. If frequencies sum to $N$, a code with lengths $\ell_i$ has total encoded length $\sum_i f_i \ell_i$ bits and average length $\sum_i (f_i/N)\ell_i$. Huffman coding minimizes this weighted path length among binary prefix codes for the supplied symbol frequencies. It does not necessarily compress every short input, because the frequency table and tree may also need to be stored or transmitted.

**Tree interpretation.** Each leaf represents one symbol; each internal node represents the merge of two lower-frequency nodes. Assigning 0 to one branch and 1 to the other produces codewords by reading the root-to-leaf path. Swapping left and right branches changes the bit patterns but not the code lengths or compression cost. If there is only one distinct symbol, an implementation should define a special convention (often a single bit) because the tree path would otherwise be empty.

**Implementation detail.** A min-heap supports efficient removal of the two smallest weights. Each merge inserts one new node. For $m$ distinct symbols there are $m - 1$ merges, so the heap-based construction takes $O(m \log m)$ time. Encoding a message then takes time proportional to the message length, assuming symbol-to-code lookup is efficient. At each step, choose the two nodes with the smallest frequencies, merge them into a new node whose frequency is their sum, and put that node back into the collection. Repeat until one tree remains. The repeated choice of the two least-frequent nodes is the key greedy rule.




Illustrative frequencies only. The two smallest nodes are merged repeatedly; the exact tree depends on the input frequencies and tie-breaking rule.

## 3.5 Python implementation

The following implementation uses Python's `heapq`, a min-heap that removes the node with the smallest frequency efficiently. Tree nodes are represented as dictionaries containing a frequency, optional symbol, and left/right children.

```python
import heapq
from itertools import count

def huffman_codes(frequencies):
    if not frequencies:
        return {}

    serial = count()
    heap = []
    for symbol, frequency in frequencies.items():
        if frequency <= 0:
            raise ValueError("Frequencies must be positive")
        node = {"symbol": symbol, "left": None, "right": None}
        heapq.heappush(heap, (frequency, next(serial), node))

    if len(heap) == 1:
        only_node = heap[0][2]
        return {only_node["symbol"]: "0"}

    while len(heap) > 1:
        freq1, _, left = heapq.heappop(heap)
```

```
        freq2, _, right = heapq.heappop(heap)
        parent = {
            "symbol": None,
            "left": left,
            "right": right
        }
        heapq.heappush(
            heap, (freq1 + freq2, next(serial), parent)
        )

    root = heap[0][2]
    codes = {}

    def build_codes(node, prefix):
        if node["symbol"] is not None:
            codes[node["symbol"]] = prefix
            return
        build_codes(node["left"], prefix + "0")
        build_codes(node["right"], prefix + "1")

    build_codes(root, "")
    return codes

frequencies = {"A": 5, "B": 9, "C": 12, "D": 13, "E": 16, "F": 45}
codes = huffman_codes(frequencies)
for symbol in sorted(codes):
    print(symbol, codes[symbol])
```

## 3.6 Code walkthrough

1. `heapq` provides a min-heap. Each heap entry contains frequency, a serial number, and a node.
2. The serial number prevents Python from needing to compare dictionary objects when two frequencies are equal.
3. Each symbol starts as a leaf node. A leaf stores the symbol; internal nodes store children.
4. `heappop` removes the two minimum-frequency nodes.
5. The new parent has frequency equal to the sum of the two removed frequencies.
6. The final remaining node is the root of the Huffman tree.
7. During traversal, appending `0` on the left and `1` on the right constructs each symbol's code.
8. A one-symbol alphabet is handled separately because the root would otherwise receive an empty code.

## 3.7 Encoding and decoding

To encode a message, replace each symbol with its code and concatenate the bits. For example, if $A \mapsto 0$ and $B \mapsto 10$, the message `ABA` becomes `0100`. This example is illustrative; actual codes depend on the complete frequency table.

To decode, start at the tree root, read one bit at a time, follow left for zero and right for one, and emit a symbol whenever a leaf is reached. Then return to the root and continue. Prefix-freeness ensures that a complete codeword can be recognized as soon as a leaf is reached.

## 3.8 Complexity

Let $n$ be the number of distinct symbols. Heap construction and the $n - 1$ merge operations take $O(n \log n)$ time overall. Tree traversal takes $O(n)$ time to assign codes, excluding the total number of output bits. Thus the standard overall construction complexity is $O(n \log n)$. The tree requires $O(n)$ nodes. Actual encoded message length also depends on symbol frequencies and code lengths.

## 3.9 Advantages, limitations, and viva

Huffman coding is lossless and adapts code lengths to observed frequencies. It does not necessarily produce the best compression for every source model, and the frequency table or tree must be available to the decoder. If all symbols have equal frequency, codes tend to have similar lengths. If one symbol dominates, its code can be very short.

1. **What is a greedy algorithm?** It repeatedly makes a locally preferred choice intended to contribute to a global solution.
2. **Why choose the two minimum frequencies?** This is the defining Huffman merge rule and yields an optimal prefix code for the supplied symbol frequencies.
3. **What is a prefix-free code?** No valid codeword is the beginning of another valid codeword.
4. **Why use a min-heap?** It efficiently retrieves the two smallest current frequencies.
5. **Is Huffman coding lossy?** No; the original symbols can be reconstructed exactly.
6. **What is the input to Huffman coding?** A set of symbols and their frequencies or weights.
7. **What is the output?** A binary codeword for each symbol.
8. **What is a leaf in the Huffman tree?** A node representing an input symbol.
9. **What does an internal node store?** Usually the combined frequency of its child nodes.
10. **Why combine the two least frequent nodes?** This greedy choice builds an optimal prefix code for the given symbol frequencies.
11. **What is a min-heap?** A priority queue that exposes the item with the smallest key.

12. **What is the usual complexity with a min-heap?** $O(n \log n)$ for $n$ symbols.
13. **What does prefix-free mean?** No codeword is a prefix of another codeword.
14. **Why is prefix-free coding useful?** The encoded stream can be decoded unambiguously.
15. **Is Huffman compression lossy?** No, it is lossless.
16. **What do 0 and 1 represent?** The two branches from a tree node, conventionally left and right.
17. **How is a symbol encoded?** Follow its root-to-leaf path and concatenate branch bits.
18. **How is the message decoded?** Traverse the tree bit by bit until reaching a symbol leaf, then restart at the root.
19. **Can two symbols have equal frequency?** Yes; tie-breaking can produce different but equally optimal codes.
20. **What must a decoder know?** The code tree or equivalent symbol-frequency/codebook information.

# 4 DAA Experiment 3: Fractional Knapsack Using a Greedy Method

## 4.1 Problem statement

A knapsack has capacity $W$. Each item $i$ has value (profit) $v_i$ and weight $w_i$. In the fractional knapsack problem, a fraction of an item may be selected. The objective is to maximize total value without exceeding the capacity:

$$\max \sum_{i=1}^{n} x_i v_i \quad \text{subject to} \quad \sum_{i=1}^{n} x_i w_i \leq W, \qquad 0 \leq x_i \leq 1.$$

The fraction $x_i$ can be any real number from zero to one. This makes a greedy strategy optimal: choose items in descending order of value per unit weight, $v_i/w_i$.

## 4.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Input item values, weights, and capacity -> compute value/weight ratios ->
    sort ratios descending -> take whole items while possible -> take a
                fraction of next item -> report maximum value
```

## 4.3 Algorithm

1. Compute each item's value-to-weight ratio.
2. Sort items by descending ratio.
3. Start with an empty knapsack and remaining capacity $W$.
4. Take a whole item if it fits.
5. Otherwise, take the fraction that fills the remaining capacity and stop.

## 4.4 Python implementation

```python
def fractional_knapsack(capacity, items):
    # Each item is a tuple: (value, weight)
    for value, weight in items:
        if weight <= 0:
            raise ValueError("Weights must be positive")

    items_sorted = sorted(
        items, key=lambda item: item[0] / item[1], reverse=True
    )

    total_value = 0.0
```

```
    chosen = []
    remaining = capacity

    for value, weight in items_sorted:
        if remaining <= 0:
            break

        if weight <= remaining:
            total_value += value
            remaining -= weight
            chosen.append((value, weight, 1.0))
        else:
            fraction = remaining / weight
            total_value += value * fraction
            chosen.append((value, weight, fraction))
            remaining = 0

    return total_value, chosen

items = [(60, 10), (100, 20), (120, 30)]
capacity = 50
value, chosen_items = fractional_knapsack(capacity, items)
print("Maximum value:", value)
print("Selected items (value, weight, fraction):", chosen_items)
```

## 4.5 Walkthrough and dry run

The ratios for the sample items are $60/10 = 6$, $100/20 = 5$, and $120/30 = 4$. The capacity is 50. Take the first item (weight 10, value 60), leaving 40. Take the second (weight 20, value 100), leaving 20. Only 20 of the final 30 weight units fit, so take fraction $20/30 = 2/3$. The final value is

$$60 + 100 + \frac{2}{3}(120) = 240.$$

The final item is not taken whole. The chosen fraction is calculated from the remaining capacity divided by the item's total weight.

## 4.6 Why the greedy rule works here

## 4.7 Proof intuition and the limit of the greedy strategy

The fractional knapsack problem has capacity $W$ and items with value $v_i$ and weight $w_i$. The decision variable $x_i$ is a fraction between zero and one: $0 \le x_i \le 1$. The optimization problem is

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \le W, \qquad 0 \le x_i \le 1.$$

Assume item weights are positive and values are non-negative. The value density $r_i = v_i/w_i$ tells us how much value is gained per unit of capacity. Sorting by decreasing density and filling in that order is optimal for the fractional version.

An exchange argument explains why. Suppose a solution uses some weight of a lower-density item while a higher-density item remains available and not fully selected. Moving a small amount of capacity from the lower-density item to the higher-density item cannot reduce value; it increases value if the densities differ. Therefore, an optimal solution can be arranged so that higher-density items are filled before lower-density items, with at most one item taken fractionally at the boundary.

The word *fractional* is essential. If an item can be divided, any leftover capacity can be filled with a fraction of the next item. In 0–1 knapsack, each decision is restricted to $x_i \in \{0, 1\}$, so the exchange argument no longer guarantees a feasible replacement. A locally attractive item can block a better combination of whole items. Greedy is not a universal optimization method; it is justified only when the problem structure supports the greedy-choice property.

**Assumptions and edge cases.** A zero-weight item with positive value needs explicit handling because $v_i/w_i$ is undefined. Negative values should generally not be selected. If total item weight is below capacity, the algorithm takes all useful items and leaves capacity unused. If capacity is zero, the total value is zero under the usual positive-weight assumption. The implementation should not silently divide by zero or assume all input values are valid.

**Practical interpretation.** The algorithm is appropriate when resources can genuinely be split, such as divisible raw materials or an allocation measured continuously. It is inappropriate when each object must be accepted or rejected as a whole, such as selecting indivisible projects or devices. Suppose a solution includes some weight from an item with a lower value-to-weight ratio while leaving available weight in a higher-ratio item. Swapping an equal amount of weight toward the higher ratio cannot decrease total value. Repeating this exchange motivates selecting the highest ratios first. The ability to take fractions is essential to this reasoning.

## 4.8 Complexity and comparison with 0–1 knapsack

Sorting dominates the runtime, so the time complexity is $O(n \log n)$ and auxiliary storage is typically $O(n)$ for the sorted collection and selected-item list. Fractional knapsack differs from 0–1 knapsack because the latter forces $x_i \in \{0, 1\}$. The ratio-greedy method is not guaranteed to solve 0–1 knapsack optimally.

## 4.9 Viva questions

1. **What is the greedy criterion?** Descending value-to-weight ratio.
2. **Can an item be partially selected?** Yes, any fraction from zero to one is allowed.
3. **Why is sorting required?** To consider the best value per unit of capacity first.
4. **What is the time complexity?** $O(n \log n)$ due to sorting.
5. **Does the same greedy strategy always solve 0–1 knapsack?** No.
6. **What is the objective?** Maximize total value without exceeding the capacity.
7. **What makes it fractional?** An item may be selected partially.
8. **What ratio guides the greedy choice?** Value divided by weight.
9. **Why sort by decreasing ratio?** It maximizes value gained per unit of capacity.

10. **What if the entire next item does not fit?** Take the fraction that fills the remaining capacity.
11. **What is the typical time complexity?** $O(n \log n)$ due to sorting.
12. **What is the space complexity?** Often $O(n)$ if item records or a sorted copy are stored.
13. **Does the greedy rule solve 0–1 knapsack optimally?** Not in general, because items cannot be split in 0–1 knapsack.
14. **What if an item has zero weight?** Handle it explicitly; positive-value zero-weight items can be selected without consuming capacity.
15. **Why check capacity remaining?** To avoid exceeding the knapsack limit.
16. **Can items be sorted by weight alone?** No; the value-to-weight ratio is the relevant greedy criterion.
17. **What happens if capacity is zero?** No positive-weight item can be taken.
18. **What is a feasible solution?** A selection whose total weight does not exceed capacity.
19. **What is a greedy-choice property?** A locally optimal choice can be part of a globally optimal solution.
20. **What should be returned?** The maximum total value and, optionally, the selected fractions.

# 5 DAA Experiment 4: 0–1 Knapsack Using Dynamic Programming

## 5.1 Problem definition

In 0–1 knapsack, each item must either be selected completely or not selected. For $n$ items with values $v_i$, weights $w_i$, and capacity $W$, maximize total value subject to the total weight not exceeding $W$. The decision variable must be $x_i \in \{0, 1\}$.

## 5.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Input values, weights, and capacity -> initialize DP table -> for each
item and capacity:  include or exclude -> fill DP table -> read optimum
                    value -> trace choices if required
```

## 5.3 Dynamic programming idea

## 5.4 Dynamic programming in detail: state, transition, and optimal substructure

Dynamic programming (DP) is useful when a problem has overlapping subproblems and optimal substructure. *Overlapping subproblems* means the same smaller questions arise repeatedly. *Optimal substructure* means an optimal solution can be built from optimal solutions to smaller states. DP avoids repeatedly solving a state by storing its answer in a table.

For 0–1 knapsack, let $DP[i][c]$ be the maximum value obtainable using only the first $i$ items and a capacity limit $c$. This definition is important: the row index specifies which items are available, and the column index specifies the capacity. The base cases are $DP[0][c] = 0$ for every capacity, because zero items yield zero value, and $DP[i][0] = 0$ for every item count, because zero capacity admits no positive-weight item.

For item $i$ with weight $w_i$ and value $v_i$, there are two possibilities. If $w_i > c$, the item cannot fit, so $DP[i][c] = DP[i-1][c]$. Otherwise, compare exclusion and inclusion:

$$DP[i][c] = \max\left(DP[i-1][c],\ v_i + DP[i-1][c-w_i]\right).$$

The exclusion term preserves the best value without item $i$. The inclusion term adds its value to the best solution for the remaining capacity using only earlier items. The previous row is used in both terms, which ensures item $i$ cannot be selected more than once.

The table is filled in increasing order of item count and capacity so that every state it references has already been computed. The answer is $DP[n][W]$. The recurrence calculates the maximum value, but it does not automatically list the chosen items. To reconstruct them, start at $DP[n][W]$ and compare each row with the row above. If the value changes, item $i$ was included; subtract $w_i$ from the capacity and continue to the previous item.

The standard table uses $O(nW)$ time and $O(nW)$ memory. This is called pseudo-polynomial time because it depends on the numeric capacity $W$, not only on the number of bits needed to represent $W$. If $W$ is extremely large, the table may be impractical even for a modest number of items. A one-dimensional optimization can reduce storage to $O(W)$, but capacities must be iterated downward; iterating upward can accidentally reuse the same item multiple times and solve an unbounded-knapsack variant instead.

**How DP differs from brute force.** Brute force checks up to $2^n$ subsets. DP combines all subsets that lead to the same state $(i, c)$, retaining only the best value for that state. It is usually much faster for moderate capacities, but it is not automatically efficient for every input size. Define $DP[i][c]$ as the maximum value achievable using only the first $i$ items with capacity $c$. For each item, either exclude it or include it if its weight fits:

$$DP[i][c] = \begin{cases} DP[i-1][c], & w_i > c, \\ \max\{DP[i-1][c], v_i + DP[i-1][c-w_i]\}, & w_i \le c. \end{cases}$$

The base cases are $DP[0][c] = 0$ for every capacity and $DP[i][0] = 0$ for every number of items. These states encode smaller versions of the same problem.

## 5.5 Python implementation

```python
def knapsack_01(capacity, weights, values):
    if len(weights) != len(values):
        raise ValueError("weights and values must have equal length")
    if capacity < 0 or any(w < 0 for w in weights):
        raise ValueError("capacity and weights must be non-negative")

    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        weight = weights[i - 1]
        value = values[i - 1]

        for current_capacity in range(capacity + 1):
            dp[i][current_capacity] = dp[i - 1][current_capacity]
            if weight <= current_capacity:
                include = value + dp[i - 1][current_capacity - weight]
                dp[i][current_capacity] = max(
                    dp[i][current_capacity], include
                )

    return dp[n][capacity], dp

weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50
best_value, table = knapsack_01(capacity, weights, values)
print("Maximum value:", best_value)
```

## 5.6 Understanding the code

The table has $(n + 1)$ rows and $(W + 1)$ columns. Row zero represents using no items; column zero represents a knapsack with no capacity. Both naturally have value zero. At row $i$, the current item is `weights[i-1]` because Python indices start at zero while the DP table reserves row zero for the base case.

For each capacity, the default is the value obtained by excluding the current item. If the item fits, calculate the value from including it and keep the larger option. Notice that the include transition uses the previous row, $DP[i-1][c-w_i]$, not the current row. This is important: it prevents the same item from being reused multiple times.

## 5.7 Complexity

The algorithm fills $(n+1)(W+1)$ states, doing constant work per state. Time complexity is $O(nW)$ and space complexity is $O(nW)$. This is called pseudo-polynomial time because it depends on the numeric capacity $W$, not just the number of bits needed to represent $W$. A one-dimensional optimization can reduce memory to $O(W)$, but capacities must be iterated downward to avoid reusing an item.

## 5.8 Fractional versus 0–1 knapsack

| Property | Fractional knapsack | 0–1 knapsack |
|---|---|---|
| Item selection | Any fraction | Whole item or none |
| Common strategy | Greedy ratio | Dynamic programming |
| Typical complexity | $O(n \log n)$ | $O(nW)$ |
| Ratio greedy optimal? | Yes | Not in general |

For the sample items $(w, v) = (10, 60), (20, 100), (30, 120)$ and capacity 50, the fractional solution reaches 240 by taking part of the third item. The 0–1 solution takes the first and third items, with weight 40 and value 180, or the first and second with weight 30 and value 160; the optimum is 220 by taking the second and third items with total weight 50 and value 220. This illustrates why the two problems differ.

## 5.9 Viva questions

1. **What does a DP state represent?** The best value for a defined prefix of items and a defined capacity.
2. **Why use the maximum of include and exclude?** These are the two legal choices for each item.
3. **What is the time complexity?** $O(nW)$.
4. **Why is the algorithm pseudo-polynomial?** Runtime depends on the numeric capacity, not only the input's bit length.
5. **Why not use the same row for the include transition?** It could accidentally allow the same item to be selected repeatedly.

6. **What does 0–1 mean?** Each item is either selected once or not selected.
7. **What does a DP state represent?** The best value using a prefix of items under a given capacity.
8. **What are the two choices for each item?** Exclude it or include it if its weight fits.
9. **What is the recurrence when weight fits?** Maximum of excluding the item and its value plus the best state at reduced capacity.
10. **What is the base case?** Zero items or zero capacity gives value zero.
11. **What is the time complexity of the table algorithm?** $O(nW)$.
12. **What is the space complexity of the full table?** $O(nW)$.
13. **Why is this pseudo-polynomial?** Runtime depends on the numeric capacity $W$, not just its encoded input length.
14. **Why does item order matter in the table?** Rows typically represent how many of the first items are available.
15. **How can selected items be recovered?** Trace backward through the table comparing adjacent rows.
16. **Why is fractional knapsack different?** Fractional knapsack allows parts of items; 0–1 knapsack does not.
17. **Can dynamic programming solve every NP-hard problem efficiently?** No; it is efficient only when the state space is manageable.
18. **What happens if item weight exceeds capacity?** That item cannot be included in states with insufficient capacity.
19. **Why use integer capacity in the classic table?** The standard table indexes discrete capacity values.
20. **What is optimal substructure?** An optimal solution can be built from optimal solutions to smaller subproblems.

# 6 Machine Learning Foundations Used in These Experiments

## 6.1 Supervised learning

## 6.2 A more complete conceptual model of machine learning

Machine learning (ML) builds a model from examples rather than requiring every prediction rule to be written manually. In supervised learning, each training example contains input features $x$ and a known target $y$. A learning algorithm selects model parameters so that predictions $\hat{y} = f_\theta(x)$ fit the training examples according to an objective or loss function. The hope is that the learned relationship generalizes to new examples drawn from a similar process.

**Regression versus classification.** Regression predicts a numerical quantity such as fare. Classification predicts a class label such as spam/not spam or churn/no churn. Some classifiers also produce a score or probability estimate. The fact that a target is encoded as 0 and 1 does not automatically make it regression: if the meaning is a category, the task is classification.

**Training, validation, and test data.** Training data are used to fit model parameters. Validation data help choose hyperparameters or monitor training, such as selecting a network epoch. Test data are reserved for a final, relatively unbiased estimate of generalization. If the same test set is repeatedly used to tune choices, it gradually becomes part of the development process and its score becomes less trustworthy. On small datasets, cross-validation may provide a more stable estimate, although the final test set must still remain separate when one is required.

**Parameters versus hyperparameters.** Parameters are learned from data, such as regression coefficients, tree split structures, SVM support vectors and weights, or neural-network weights and biases. Hyperparameters are choices made before or around training, such as the number of neighbors $K$, the SVM kernel and regularization setting, the number of trees, hidden-layer widths, learning rate, and number of epochs. Hyperparameters should be chosen using training/validation procedures, not by optimizing directly against the final test score.

**Generalization and overfitting.** A model overfits when it learns patterns specific to the training data, including noise, that do not transfer to unseen data. Training performance may then be strong while test performance is weak. Underfitting occurs when the model is too simple or insufficiently trained to capture useful patterns. The goal is not to memorize the training set; it is to minimize expected error on new cases. More complex models, more features, or longer training do not guarantee better generalization.

**Data leakage.** Leakage occurs when information unavailable at prediction time, or information from the evaluation data, influences training. Examples include fitting a scaler on the full dataset before splitting, using a post-outcome column to predict the outcome, or selecting features after inspecting test labels. Leakage often creates unrealistically high scores. The safe principle is to split first, fit preprocessing on training data only, and apply the fitted transformation unchanged to validation and test data.

**Class imbalance.** If only a small fraction of bank customers churn, a model predicting "no churn" for everyone can have high accuracy while failing to identify the customers of

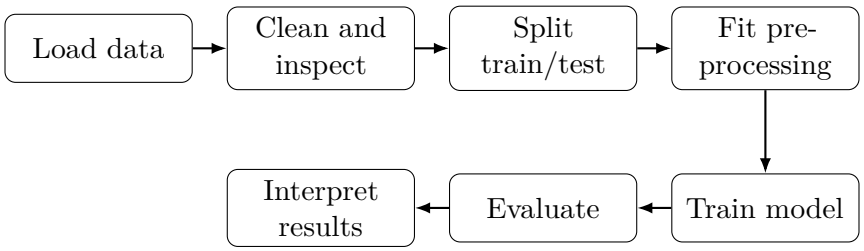
interest. Always examine class counts and metrics such as precision, recall, F1-score, and the confusion matrix. The appropriate metric depends on the cost of false positives and false negatives in the application. Supervised learning learns a mapping from input features $X$ to a known target $y$ using labelled examples. In **regression**, the target is numeric, such as a fare amount. In **classification**, the target is a category, such as spam versus not spam or churn versus no churn.

A dataset is commonly divided into:

- **Features ($X$):** the columns used to make predictions.
- **Target ($y$):** the value the model is intended to predict.
- **Training set:** used to learn model parameters.
- **Test set:** held back until evaluation to estimate performance on unseen examples.

A test set must not influence fitting or preprocessing decisions. Using test information during training creates data leakage and can make reported performance overly optimistic.

## 6.3 Typical ML pipeline




For numerical scaling, fit the scaler on training data only. For categorical encoding, ensure train and test columns are aligned. For any data-dependent transformation, determine whether fitting it before the split would expose information from the test set.

## 6.4 Common preprocessing operations

- **Missing values:** inspect with `isnull().sum()`; either remove rows/columns or impute using a justified rule.
- **Categorical encoding:** convert text categories into numeric representation. One-hot encoding is appropriate for nominal categories without a natural order.
- **Scaling:** standardization transforms a feature using $z = (x - \mu)/\sigma$. Min-max scaling maps values into a specified interval, often $[0, 1]$.
- **Outliers:** inspect extreme or implausible values. Do not remove data solely because it is unusual; first consider domain meaning and data quality.
- **Train/test split:** a common starting point is 80% training and 20% testing. Classification may use `stratify=y` to preserve class proportions.

## 6.5 Reproducibility

A fixed `random_state` makes the split reproducible for the same data and software behavior. It does not guarantee identical results across every model, hardware, or library version. Results should include the data source, target definition, preprocessing choices, model settings, and evaluation measures.

# 7 ML Experiment 1: Uber Fare Prediction with Linear Regression and Random Forest

## 7.1 Aim and dataset

Predict an Uber ride's fare from pickup and drop-off information. The laboratory list names the Kaggle Uber fares dataset. A commonly used file is `uber.csv`, with fields such as `fare_amount`, pickup datetime, pickup longitude/latitude, drop-off longitude/latitude, and passenger count. Column names and exact rows can vary by dataset version; inspect the actual CSV before running the program.

This is a **regression** problem because fare amount is numeric. The target is typically `fare_amount`; coordinates and passenger count are candidate features. Identifiers, timestamps, and invalid coordinates need careful treatment.

## 7.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Load Uber data -> inspect and clean -> identify outliers -> analyze
correlation -> prepare features and target -> split train/test -> train
Linear Regression and Random Forest -> evaluate R-squared / RMSE ->
compare
```

## 7.3 Linear regression theory

## 7.4 Linear regression: assumptions, fitting, and interpretation

For $p$ input features, linear regression predicts a target as $\hat{y} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$. The intercept $\beta_0$ is the predicted value when all numeric inputs are zero, while coefficient $\beta_j$ represents the model's predicted change in $y$ for a one-unit increase in feature $x_j$, holding the other included features fixed. Interpretation depends on the feature units and whether the model specification is sensible.

Ordinary least squares chooses coefficients to minimize the residual sum of squares, $RSS = \sum_i (y_i - \hat{y}_i)^2$. Squaring makes large errors contribute more strongly and produces a convenient optimization objective. Linear regression is a model family, not a statement that the real-world relationship must be perfectly linear. Curved effects can sometimes be represented by adding transformations such as squared distance or interaction features, while the model remains linear in its coefficients.

Important assumptions for classical statistical inference include an adequately specified linear relationship in the parameters, independent errors, constant error variance, and conditions on error distributions for exact small-sample inference. Prediction can still be useful when some assumptions are imperfect, but violations can make coefficient interpretation and uncertainty estimates unreliable. Correlated predictors can make coefficients unstable, and outliers can have disproportionate influence because residuals are squared.

For Uber fares, longitude and latitude are not automatically a good representation of travel distance. A linear model using raw coordinates may learn broad location effects but does not directly encode road distance, route, traffic, or time-of-day effects. A distance feature, pickup-time features, and careful handling of impossible coordinates may improve the feature representation. Any such feature must be calculated from information available when the fare is predicted.

**Strengths and limitations.** Linear regression is fast, serves as a useful baseline, and is relatively interpretable. It may underfit nonlinear relationships, can be sensitive to extreme observations, and may extrapolate implausibly outside the training range. It should be compared using held-out data rather than assumed inferior to a more complex method. Linear regression models a numeric target as a weighted combination of features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

The parameters are fitted to reduce prediction error, commonly the sum of squared residuals. A residual is the difference between an observed value and its prediction. Linear regression is relatively interpretable but may not capture nonlinear relationships unless suitable features are engineered.

## 7.5 Random forest regression theory

## 7.6 Random forests: how tree ensembles reduce variance

A decision tree divides the feature space into regions using sequential split rules. For regression, a split is typically chosen to reduce the variation of target values within child nodes, and a leaf predicts a representative value such as the mean of training targets in that leaf. Trees can model nonlinear relationships and interactions, but a single deep tree can be unstable: a small change in training data may change its structure substantially.

A random forest combines many trees. Each tree is trained on a bootstrap sample, which is a sample drawn from the training data with replacement. At each split, only a random subset of candidate features is considered. These sources of randomness make trees less correlated. Averaging predictions from diverse trees often reduces variance compared with relying on one tree. For regression, the forest prediction is commonly $\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$, where $B$ is the number of trees and $T_b$ is the prediction of tree $b$.

Random forests can model nonlinear effects without requiring the user to specify a formula and generally need less feature scaling than distance-based models or neural networks. However, they may be slower and larger than a simple linear model, are less straightforward to interpret globally, and do not extrapolate smoothly beyond patterns represented in training data. More trees can stabilize predictions but increase computation. Depth, minimum leaf size, feature subsampling, and number of trees affect the bias-variance and runtime trade-offs.

**Why compare it with linear regression?** The two models make different assumptions. Linear regression is a simple parametric baseline; a random forest is a flexible non-parametric ensemble. If the forest performs better on test data, that suggests nonlinearities or interactions may matter, but the result does not prove causation or guarantee future superiority. If the linear model performs similarly, its simplicity and interpretability may make it preferable. A random forest combines predictions from many decision trees. Each tree is trained on a bootstrap sample and considers a randomized subset of features when splitting. For regression, predictions are generally averaged across trees. The ensemble can capture

nonlinear interactions without requiring a linear equation, but it is less directly interpretable and may require more computation. It is not guaranteed to outperform linear regression on every dataset.

## 7.7 Outlier identification and correlation

The interquartile range is $IQR = Q_3 - Q_1$. A common rule flags values below $Q_1 - 1.5IQR$ or above $Q_3 + 1.5IQR$ as potential outliers. This is a screening heuristic, not proof that the row is wrong. For fares, negative or implausibly high amounts and invalid coordinates deserve inspection.

Correlation measures association between variables. Pearson correlation ranges from $-1$ to 1: positive values indicate positive linear association, negative values indicate negative linear association, and values near zero indicate weak linear association. Correlation does not prove causation and may miss nonlinear relationships. Correlation analysis should not be performed on arbitrary encoded identifiers as though they were meaningful numeric quantities.

## 7.8 Complete example code

This version uses common column names and includes basic validation. Install dependencies if needed with `pip install pandas scikit-learn numpy`. The dataset itself must be obtained from the source named by the laboratory.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

# 1. Load and inspect
df = pd.read_csv("uber.csv")
print(df.head())
print(df.info())
print(df.isnull().sum())

# 2. Clean essential missing values
required = [
    "fare_amount", "pickup_longitude", "pickup_latitude",
    "dropoff_longitude", "dropoff_latitude", "passenger_count"
]
df = df.dropna(subset=required)

# 3. Basic validity filters; adapt thresholds to the dataset/region
df = df[df["fare_amount"] > 0]
df = df[df["passenger_count"] > 0]
df = df[
    df["pickup_longitude"].between(-180, 180) &
    df["dropoff_longitude"].between(-180, 180) &
```

```
    df["pickup_latitude"].between(-90, 90) &
    df["dropoff_latitude"].between(-90, 90)
]

# 4. IQR screening for fare outliers
q1 = df["fare_amount"].quantile(0.25)
q3 = df["fare_amount"].quantile(0.75)
iqr = q3 - q1
lower = max(0, q1 - 1.5 * iqr)
upper = q3 + 1.5 * iqr
df = df[df["fare_amount"].between(lower, upper)]

# 5. Features and target
features = [
    "pickup_longitude", "pickup_latitude",
    "dropoff_longitude", "dropoff_latitude", "passenger_count"
]
X = df[features]
y = df["fare_amount"]

# 6. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 7. Train models
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

forest_model = RandomForestRegressor(
    n_estimators=100, random_state=42, n_jobs=-1
)
forest_model.fit(X_train, y_train)

# 8. Predict and evaluate
for name, model in [
    ("Linear Regression", linear_model),
    ("Random Forest", forest_model)
]:
    predictions = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, predictions))
    print(name)
    print("R2:", r2_score(y_test, predictions))
    print("RMSE:", rmse)
```

## 7.9 Code explanation

1. `read_csv` loads the CSV into a pandas DataFrame. `head`, `info`, and `isnull().sum()` reveal the structure and missingness.

2. `dropna(subset=required)` removes rows missing essential values. This is one possible

strategy; imputation may be better if many rows are missing.

3. Validity filters remove impossible coordinate ranges, nonpositive fares, and nonpositive passenger counts. Geographic plausibility and fare limits should be adapted to the actual data.
4. IQR filtering flags fare values outside the conventional range. Removing such values changes the population being modelled, so document the choice and compare with an unfiltered baseline if appropriate.
5. `X` contains predictors and `y` contains the target. The target must not accidentally appear among the features.
6. `train_test_split` reserves a held-out test set. `random_state=42` makes the split reproducible.
7. `fit` learns model parameters from the training set.
8. `predict` generates fares for test rows; the metrics compare these predictions with the true fares.

## 7.10 Evaluation measures

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

An $R^2$ closer to one often indicates better fit on the evaluated data; zero roughly corresponds to predicting the test-set mean under the standard definition, and negative values indicate performance worse than that baseline. It is not a percentage accuracy score.

Root mean squared error is

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}.$$

RMSE is in the target's units and penalizes large errors more heavily due to squaring. Lower RMSE is generally better when models are evaluated on the same test set. Compare both measures; do not assume a random forest always wins.

## 7.11 Practical caveats

Latitude and longitude are not direct distance measurements. A better feature set may include haversine distance, pickup hour/day, and other permitted information. If a pickup datetime is used, parse it into meaningful time features instead of feeding an arbitrary text string into the model. Be careful not to use information unavailable at prediction time. A random train/test split may not reflect future deployment if fares are subject to temporal drift; a time-based split may be more realistic.

## 7.12 Viva questions

1. **Why is fare prediction regression?** Because the target is a continuous numeric amount.
2. **What does `fit()` do?** It estimates the model from training data.
3. **What does $R^2$ measure?** How much better predictions fit relative to a mean-prediction baseline under the metric's definition.
4. **Why use RMSE?** It measures typical error magnitude in the target units and penalizes large errors.
5. **What is the IQR rule?** A heuristic for identifying values far from the middle 50% of observations.
6. **Why compare two regressors?** They represent different assumptions and can perform differently on the same data.
7. **Why is fare prediction a regression task?** The target fare is a continuous numeric value.
8. **What is the target column?** The fare amount column, commonly named `fare_amount`.
9. **What is a feature?** An input variable used to predict the target.
10. **Why remove invalid fares?** They can be data errors or outside the intended prediction scope.
11. **What is an outlier?** An observation unusually distant from the bulk of the data under a chosen rule.
12. **What does the IQR rule use?** The first and third quartiles and the interquartile range.
13. **What does correlation measure?** The strength and direction of association between variables, often linear correlation.
14. **Does correlation prove causation?** No.
15. **What does Linear Regression learn?** Coefficients and an intercept that minimize a fitting loss, commonly squared error.
16. **What is a Random Forest?** An ensemble of decision trees whose regression predictions are averaged.
17. **Why split train and test data?** To estimate performance on observations not used for fitting.
18. **What is RMSE?** The square root of mean squared prediction error, in target units.
19. **What is MAE?** The mean absolute difference between actual and predicted values.
20. **What does $R^2$ indicate?** Fit relative to a mean-prediction baseline; it can be negative on test data.

# 8 ML Experiment 2: Email Spam Detection Using KNN and SVM

## 8.1 Aim and dataset

Classify an email as normal (not spam) or spam using binary classification, and compare K-Nearest Neighbors and Support Vector Machine models. The laboratory list points to the Kaggle `emails.csv` dataset. A commonly used variant contains 5,172 rows and 3,002 columns: an email identifier, 3,000 word-frequency features, and a `Prediction` target. Confirm the real column names and label meaning in the supplied file before running the code.

For the common version, `Prediction`=1 means spam and `Prediction`=0 means not spam. The identifier column is not a content feature and should normally be removed. The word-frequency columns are numeric but may have very different scales or distributions.

## 8.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Load emails -> inspect labels and features -> clean/encode data -> split
train/test -> scale features where needed -> train KNN and SVM -> predict
       spam/ham -> compute confusion matrix and metrics -> compare
```

## 8.3 K-Nearest Neighbors

## 8.4 KNN in depth: distance, scaling, and the choice of $K$

K-Nearest Neighbors is an instance-based or lazy-learning method. It does not fit a compact global equation during training; instead, it stores the training examples and uses them when a new example arrives. For a query point, it calculates distances to training points, selects the $K$ nearest, and predicts from their labels. In classification, the basic rule is majority vote; distance-weighted voting gives closer neighbors greater influence.

For numeric vectors, Euclidean distance is $d(x, z) = \sqrt{\sum_{j=1}^{p}(x_j - z_j)^2}$. If one feature is measured in thousands and another in fractions, the large-scale feature can dominate this distance even if it is not more informative. Standardization or another appropriate scaling method is therefore often essential for KNN. Scaling must be fit on training data and then applied to test data with the same fitted parameters.

The hyperparameter $K$ controls locality. A very small $K$ can be sensitive to noise and individual training points (high variance); a very large $K$ smooths the decision boundary and can blur minority-class regions (higher bias). An odd $K$ can reduce ties in binary classification, but it does not guarantee that ties are impossible in all multiclass or weighted settings. The best value should be selected on validation data or through cross-validation.

Prediction can be expensive because a basic implementation compares each query with many or all training examples. With $n$ training samples and $p$ features, a straightforward query has roughly $O(np)$ distance work, excluding sorting or selection costs. KNN is easy to explain and can model irregular boundaries, but it needs memory for training data and may perform

poorly with many irrelevant features or high-dimensional sparse vectors. This is the curse of dimensionality: distances become less discriminative as dimension grows.

For email spam data with thousands of word-frequency features, KNN can be computationally demanding and feature scaling can materially change which emails are considered similar. The representation and metric are as important as the value of $K$. KNN is an instance-based classifier. During fitting, it stores the training examples. To classify a new email, it computes distances from that email to training examples, selects the $K$ nearest examples, and predicts the majority class among those neighbors. A small $K$ can be sensitive to noise; a large $K$ smooths the decision but can overlook local patterns.

Euclidean distance between vectors $x$ and $z$ is

$$d(x, z) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}.$$

Scaling matters because features with larger numeric ranges can dominate distance. KNN prediction can be computationally expensive on large datasets because distances to many training examples may be calculated for each query.

## 8.5 Support Vector Machine

## 8.6 SVM in depth: margins, support vectors, and kernels

A Support Vector Machine (SVM) learns a decision boundary that separates classes while seeking a large margin between the boundary and the closest training examples. For a linear classifier, the decision function can be written $f(x) = w^T x + b$, with the predicted class determined by the sign of the score. The margin is related to the distance from the boundary to the nearest examples. A larger margin can improve generalization, although the final result depends on data quality, regularization, and the selected kernel.

The training examples closest to the boundary are called support vectors; they play a central role in determining the classifier. Real data may not be perfectly separable, so a soft-margin SVM allows some margin violations and balances classification errors against margin size. In common implementations, parameter $C$ controls this trade-off: a larger $C$ penalizes training violations more strongly, while a smaller $C$ allows a wider margin at the cost of more training errors.

A linear kernel is suitable when a linear decision boundary is reasonable and can be efficient for high-dimensional sparse text representations. A nonlinear kernel, such as the radial basis function (RBF) kernel, implicitly represents nonlinear similarity and can create curved boundaries. Nonlinear kernels can require more computation and careful hyperparameter tuning. Kernel choice should be validated rather than selected by habit.

SVMs are sensitive to feature scales because distances and dot products influence the margin and kernel values. Standardization is commonly applied to numerical features. For large text datasets, a linear SVM is often a practical baseline, but the exact performance depends on the dataset, feature representation, and regularization. SVM output scores are not automatically calibrated probabilities; a separate probability-estimation procedure may be needed if calibrated probabilities are required.

**KNN versus SVM.** KNN delays computation until prediction and depends on local neighborhoods. SVM learns a boundary during training and depends on margin optimization.

KNN's main tuning parameter is $K$ and its distance metric; SVM choices include kernel and regularization. Both can benefit from scaling, but neither should be declared universally better without a fair held-out comparison. An SVM aims to find a decision boundary with a large margin between classes. A linear SVM uses a linear decision function; kernels can represent nonlinear boundaries. The regularization parameter $C$ controls the trade-off between a wider margin and training errors: larger $C$ penalizes training errors more strongly, which may increase overfitting risk. Feature scaling is important for SVM because its optimization and distances/margins depend on feature magnitudes.

## 8.7 Complete example code

```python
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# 1. Load and inspect the dataset
df = pd.read_csv("emails.csv")
print(df.head())
print(df.shape)
print(df.columns[:10])

# 2. Remove identifier-like column if present
df = df.drop(columns=["Email No."], errors="ignore")

# 3. Separate features and target
target = "Prediction"
if target not in df.columns:
    raise ValueError("Expected target column 'Prediction'; inspect CSV columns.
    ")
X = df.drop(columns=[target])
y = df[target]

# 4. Split with class proportions preserved
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 5. Scale features: fit only on training data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Train classifiers
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train_scaled, y_train)

svm = SVC(kernel="linear", C=1.0)
svm.fit(X_train_scaled, y_train)

# 7. Evaluate both models
for name, model in [("KNN", knn), ("Linear SVM", svm)]:
    y_pred = model.predict(X_test_scaled)
    print("\\n", name)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion matrix:\\n", confusion_matrix(y_test, y_pred))
    print(classification_report(y_test, y_pred, zero_division=0))
```

## 8.8 Explanation of the pipeline

1. Load the CSV and inspect its dimensions. A shape of $(5172, 3002)$ is common for this dataset variant, not guaranteed for every file.
2. Drop the identifier column if present. `errors="ignore"` allows the code to continue if the name is absent; always inspect the columns so the correct identifier is removed.
3. Set $X$ to the word-frequency features and $y$ to the spam label.
4. Use `stratify=y` so the train and test splits approximately preserve the original class proportions.
5. `fit_transform` learns mean and standard deviation from training data and scales it. `transform` applies those same training statistics to test data. Fitting the scaler on the whole dataset would leak test information.
6. `n_neighbors=5` means KNN considers five nearest training examples.
7. `kernel="linear"` asks the SVM to use a linear decision boundary. `C=1.0` sets the regularization strength.
8. Accuracy, confusion matrix, precision, recall, and F1 help evaluate errors beyond one headline number.

## 8.9 Confusion matrix and classification metrics

For binary classification, the confusion matrix is often written as

| | Predicted 0 | Predicted 1 |
|---|---|---|
| Actual 0 | $TN$ | $FP$ |
| Actual 1 | $FN$ | $TP$ |

where $TN$ is true negative, $FP$ false positive, $FN$ false negative, and $TP$ true positive. With spam as class 1, a false positive is a normal email incorrectly marked spam; a false negative is spam that reaches the normal inbox.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$
$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision answers: among emails predicted spam, how many were truly spam? Recall answers: among all actual spam emails, how many did the model identify? F1 balances precision and recall. When classes are imbalanced, accuracy alone can be misleading.

## 8.10 Comparing KNN and SVM

KNN has a simple training stage but can have slower prediction, stores examples, and is sensitive to feature scaling and the choice of $K$. A linear SVM may be effective in high-dimensional feature spaces, but training can be expensive depending on dataset size and implementation. Which model is better must be decided from the same held-out test set and relevant metrics. If missing spam is especially costly, recall may be more important; if legitimate email being blocked is especially costly, precision and false-positive rate deserve attention.

## 8.11 Viva questions

1. **What is binary classification?** Predicting one of two classes.
2. **What does $K = 5$ mean in KNN?** The five nearest training samples vote on the predicted class.
3. **Why scale features?** To prevent feature magnitudes from dominating distance-based or margin-based learning.
4. **What is an SVM margin?** The separation distance around a decision boundary that the classifier seeks to maximize.
5. **What is a false negative in spam detection?** An actual spam email predicted as not spam.
6. **Why is recall not the same as precision?** Recall uses all actual positives as its denominator; precision uses all predicted positives.
7. **Is spam detection classification or regression?** Binary classification.
8. **What are the two classes?** Spam and not-spam (ham).
9. **Why encode text or categories?** Most scikit-learn estimators require numeric feature representations.
10. **What is TF-IDF?** A representation that weights terms by local frequency and corpus-wide rarity.
11. **How does KNN classify an email?** It uses the labels of the nearest training examples, typically by majority vote.
12. **Why scale features for KNN?** KNN relies on distance calculations.
13. **What is a support vector?** A training point that helps define the SVM decision boundary.

14. **What is the SVM margin?** The separation region between the decision boundary and closest class points.
15. **What is a kernel?** A function enabling a model to represent nonlinear decision boundaries through transformed similarities.
16. **What is a false positive in spam detection?** A legitimate email incorrectly marked as spam.
17. **What is a false negative?** A spam email incorrectly classified as legitimate.
18. **Why may precision matter?** It reflects how many messages flagged as spam truly are spam.
19. **Why may recall matter?** It reflects how much actual spam is detected.
20. **What does a confusion matrix show?** Counts of actual versus predicted classes.

# 9 ML Experiment 3: Bank Customer Churn Prediction Using an ANN

## 9.1 Aim and dataset

Given a bank customer, build a neural-network classifier to predict whether the customer will leave within the next six months. The laboratory description refers to a Kaggle bank customer churn dataset with 10,000 sample points and 14 features, including CustomerId, CreditScore, Geography, Gender, Age, Tenure, Balance, and related customer attributes.

A common Kaggle version is named `Churn_Modelling.csv`, with target column `Exited`. It may contain columns such as `RowNumber`, `CustomerId`, and `Surname`. These are common-version assumptions: inspect the actual CSV and adapt filenames and column names if your instructor's version differs. In the common dataset, `Exited`=1 indicates churn and `Exited`=0 indicates that the customer stayed.

## 9.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Load churn data -> select features and target -> encode categorical
columns -> split train/test -> scale numeric data -> build ANN -> compile
-> train -> predict -> evaluate accuracy and confusion matrix
```

## 9.3 Artificial Neural Network theory

## 9.4 Artificial neural networks: units, layers, and learned parameters

An Artificial Neural Network (ANN) is a parameterized function built from connected computational units called neurons. In a fully connected layer, each output neuron receives every value from the preceding layer. A neuron first forms a weighted sum and bias, $z = \sum_j w_j x_j + b$, then applies an activation function $a = \phi(z)$. Weights control the influence of inputs; biases shift the activation threshold. These values are learned from training data.

A feed-forward network typically contains an input representation, one or more hidden layers, and an output layer. The input layer corresponds to the feature vector; it is not usually counted as a layer with trainable transformations. Hidden layers learn intermediate representations. The output layer's size and activation depend on the task. For binary classification, one output neuron with sigmoid activation can represent the estimated probability of class 1. For multiclass single-label classification, a common design uses one output per class with softmax. For regression, a linear output is often appropriate.

The ReLU activation is $\mathrm{ReLU}(z) = \max(0, z)$. It is computationally simple and introduces nonlinearity, allowing stacked layers to represent relationships beyond a single linear transformation. The sigmoid function $\sigma(z) = 1/(1 + e^{-z})$ maps a real number to $(0, 1)$ and is often used at the binary output. An output of 0.82 should be interpreted as the model's estimated probability or score for class 1, not a guarantee that the customer will churn.

A network without nonlinear hidden activations would collapse into a single linear transformation, no matter how many dense layers were stacked. Nonlinear activation functions are therefore essential for learning complex patterns. However, increasing the number of layers or neurons also increases the number of parameters and can cause overfitting when the dataset is limited.

**Scaling matters.** Neural-network optimization is often easier when numeric input features have comparable scales. A feature such as balance may have values in thousands, while a binary indicator has values 0 or 1. Without scaling, gradients and optimization can behave poorly. Categorical variables such as geography and gender must also be encoded numerically in a way that preserves their meaning; arbitrary integer labels can accidentally imply an order that does not exist.

**ANN versus traditional methods.** Neural networks can learn flexible interactions but require choices about architecture, optimization, regularization, and training duration. They may need more data and computation than simpler models. A useful practical baseline and careful test evaluation remain important even when an ANN is the required experiment. An Artificial Neural Network consists of connected computational units called neurons, organized into layers:

- **Input layer:** receives the feature values.
- **Hidden layer(s):** learn intermediate representations.
- **Output layer:** produces the model's prediction.

A dense layer connects each neuron to all outputs of the preceding layer. A neuron computes a weighted sum plus a bias, then applies an activation function:

$$z = \sum_{j=1}^{p} w_j x_j + b, \qquad a = \phi(z).$$

Weights and biases are learned during training. ReLU, $\mathrm{ReLU}(z) = \max(0, z)$, is a common hidden-layer activation. Sigmoid, $\sigma(z) = 1/(1 + e^{-z})$, maps a value to the interval $(0, 1)$ and is commonly used for a binary-classification output interpreted as a probability.

## 9.5 Forward propagation, loss, and backpropagation

## 9.6 Training mathematics: forward pass, loss, gradients, and optimization

During forward propagation, the feature vector passes through each layer. Every layer calculates weighted sums and activations until the network produces a prediction $\hat{y}$. A loss function then measures disagreement between prediction and target. Training aims to find weights and biases that minimize the average loss over training examples.

For binary classification with target $y \in \{0, 1\}$ and predicted probability $p$, binary cross-entropy is

$$L(y, p) = -\left[y \log(p) + (1 - y) \log(1 - p)\right].$$

If the true label is 1, the loss penalizes low $p$; if the true label is 0, it penalizes high $p$. This makes the loss sensitive to confident, incorrect predictions. Implementations numerically stabilize calculations to avoid taking logarithms of exactly zero.

Backpropagation applies the chain rule of calculus to compute how the loss changes with each weight and bias. These derivatives are gradients. An optimizer uses the gradients to update the parameters. A simple gradient-descent update is $\theta \leftarrow \theta - \eta \nabla_\theta L$, where $\eta$ is the learning rate. Adam adapts updates using moving estimates of gradient means and squared gradients; it is a widely used default, but its success is not guaranteed for every dataset or hyperparameter choice.

An epoch is one complete pass through the training portion supplied to the fitting procedure. With mini-batch training, the data are divided into batches; the optimizer updates parameters after processing each batch. For $N$ training rows and batch size $B$, the number of batches per epoch is approximately $\lceil N/B \rceil$. More epochs give the optimizer more opportunities to learn, but validation performance can eventually stop improving or deteriorate due to overfitting.

A validation split reserves part of the training data to monitor performance during fitting. It is not the same as the final test set. If validation loss increases while training loss continues to decrease, overfitting may be occurring. Early stopping, dropout, weight regularization, fewer hidden units, or more data may help, but the appropriate remedy depends on evidence rather than assumption.

The output probability must be converted into a decision if discrete labels are required. A threshold of 0.5 predicts class 1 when $p \geq 0.5$ and class 0 otherwise. That threshold is a convention, not a law of nature. When false negatives and false positives have different costs, the threshold can be tuned using validation data. The test set should be used only for the final evaluation of the selected model and threshold. During forward propagation, input features pass through the layers to produce a prediction. Binary cross-entropy measures the discrepancy between a true binary label $y$ and predicted probability $p$:

$$L = -\left[y \log(p) + (1 - y) \log(1 - p)\right].$$

Across a batch, the loss is averaged or otherwise aggregated. Backpropagation computes gradients of the loss with respect to weights and biases using the chain rule. An optimizer such as Adam uses those gradients to update the parameters. An **epoch** is one pass through the training data; a **batch** is the subset processed for one update.

## 9.7 Complete example code

The program below follows the common Kaggle column names. TensorFlow/Keras and the standard scientific Python packages must be installed in the environment. If the data file uses different names, inspect `dataset.columns` and modify the target, categorical, and identifier columns accordingly.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

```

```
# 1. Load and inspect
dataset = pd.read_csv("Churn_Modelling.csv")
print(dataset.head())
print(dataset.shape)
print(dataset.isnull().sum())
print(dataset.columns.tolist())

# 2. Remove rows with missing values (simple baseline strategy)
dataset = dataset.dropna()

# 3. Remove identifier/name columns if present
dataset = dataset.drop(
    columns=["RowNumber", "CustomerId", "Surname"],
    errors="ignore"
)

# 4. One-hot encode categorical features if present
categorical_columns = [
    col for col in ["Geography", "Gender"] if col in dataset.columns
]
dataset = pd.get_dummies(
    dataset, columns=categorical_columns,
    drop_first=True, dtype=int
)

# 5. Separate features and target
target = "Exited"
if target not in dataset.columns:
    raise ValueError("Expected target 'Exited'; inspect the dataset.")
X = dataset.drop(columns=[target])
y = dataset[target]

# 6. Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y
)

# 7. Standardize using training statistics only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 8. Build the neural network
model = Sequential()
model.add(Dense(
    16, activation="relu",
    input_shape=(X_train_scaled.shape[1],)
))
model.add(Dense(8, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

# 9. Configure learning
```

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# 10. Train
model.fit(
    X_train_scaled, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# 11. Predict probabilities and convert to class labels
y_prob = model.predict(X_test_scaled)
y_pred = (y_prob >= 0.5).astype(int).ravel()

# 12. Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
```

## 9.8 Preprocessing explained

1. `read_csv` loads the data. `head()` previews rows, `shape` reports rows and columns, and `isnull().sum()` counts missing entries.
2. `dropna()` removes rows containing missing values. This simple baseline can be wasteful if many values are missing; imputation may be preferable in a more careful analysis.
3. Identifiers and names are removed because they generally do not represent transferable churn patterns. Whether any column should be removed depends on the data definition and prediction objective.
4. One-hot encoding converts nominal categories such as geography and gender into indicator columns. `drop_first=True` omits one reference category per encoded feature group, reducing redundant columns for many models. Neural networks can also use full one-hot encoding.
5. $X$ contains input features and $y$ is the target. The target must never be included in $X$.
6. `test_size=0.2` reserves 20% of the data for testing. `stratify=y` helps preserve the target-class proportions in both sets.
7. The scaler is fitted only on training data. Test data is transformed using the training mean and standard deviation, not statistics calculated from the test set.

## 9.9 Understanding the network definition

```
model = Sequential()
model.add(Dense(16, activation="relu",
                input_shape=(X_train_scaled.shape[1],)))
model.add(Dense(8, activation="relu"))
model.add(Dense(1, activation="sigmoid"))
```

`Sequential()` stacks layers in order. The first dense layer has 16 neurons and expects one input per feature column. `X_train_scaled.shape[1]` is the number of feature columns after encoding. The second hidden layer has 8 neurons. The output layer has one neuron because the target is binary. Sigmoid produces a probability-like output between zero and one.

## 9.10 Compile, fit, predict, and evaluate

- `compile()` configures how the network learns. Adam is the optimizer, binary cross-entropy is the loss function, and accuracy is a metric tracked during training.
- `fit()` performs training. `epochs=20` requests 20 passes through the training subset. `batch_size=32` uses 32 examples per batch update. `validation_split=0.1` sets aside 10% of the provided training data for validation; it is not the held-out test set.
- `predict()` returns estimated probabilities. The threshold of 0.5 maps probabilities at least 0.5 to class 1 and smaller probabilities to class 0.
- `accuracy_score` calculates the fraction of correct labels. The confusion matrix and classification report help reveal which type of churn error is occurring.

## 9.11 Churn-specific interpretation and improvements

A false negative means the model predicts that a customer will stay, but the customer actually churns. A false positive means the model predicts churn, but the customer stays. For a retention team, failing to identify a customer who will leave may be costly, so recall for the churn class may be important. However, high recall may create more false positives and costly outreach. The decision threshold should reflect business costs, not be treated as universally fixed at 0.5.

Potential improvements include:

- Compare against a simple baseline and classical models such as logistic regression or random forest.
- Track precision, recall, F1, ROC-AUC, and a confusion matrix, not accuracy alone.
- Use early stopping or a validation-based tuning process to reduce overfitting.
- Address class imbalance with suitable class weights, resampling, or threshold tuning, applied only within the training process.
- Tune layer sizes, learning rate, batch size, epochs, and regularization on validation data rather than the test set.
- Keep the test set untouched until model selection is complete.

## 9.12 Viva questions

1. **What is churn?** A customer discontinuing the relationship or service, according to the dataset's target definition.
2. **Why is sigmoid used in the output layer?** It maps the output to a value between zero and one for binary classification.
3. **What is ReLU?** The activation function $\max(0, z)$, commonly used in hidden layers.
4. **What is an epoch?** One complete pass through the training data used by the training procedure.
5. **What does backpropagation do?** Computes gradients of the loss with respect to model parameters.
6. **What does Adam do?** Uses gradient information and adaptive moment estimates to update model parameters.
7. **What is a false negative for churn?** The model predicts that the customer will stay, but the customer actually churns.
8. **What is recall for class 1?** The proportion of actual churners correctly identified by the model.
9. **What does churn mean?** A customer leaving or closing the service relationship within the defined period.
10. **Why is churn prediction classification?** The target indicates churn versus non-churn.
11. **What is an ANN?** A model made of interconnected units arranged in layers.
12. **What does an input layer do?** Receives the feature vector.
13. **What do hidden layers do?** Learn intermediate nonlinear representations.
14. **What does a sigmoid output represent for binary classification?** A value interpretable as the model's estimated probability for the positive class when trained appropriately.
15. **Why encode categorical columns?** Neural-network inputs must be numeric.
16. **Why scale numeric inputs?** It can improve optimization stability and training behaviour.
17. **What is an epoch?** One pass through the training dataset.
18. **What is a batch?** A subset of training examples used for one gradient update.
19. **What is a loss function?** A scalar objective measuring prediction error during training.
20. **What is an optimizer?** The method used to update learned parameters from gradients.

# 10 Cross-Experiment Comparison and Exam Recall Tables

## 10.1 DAA strategies

| Experiment | Main strategy | Key decision | Typical complexity |
|---|---|---|---|
| Fibonacci | Recursion / iteration | Solve smaller index or update prior values | Recursive $O(2^n)$; iterative $O(n)$ |
| Huffman | Greedy + min-heap | Merge two minimum frequencies | $O(n \log n)$ |
| Fractional knapsack | Greedy | Highest value/weight first | $O(n \log n)$ |
| 0–1 knapsack | Dynamic programming | Include or exclude each item | $O(nW)$ |
| N-Queens | Backtracking | Place a safe queen, undo on failure | Exponential search; often bounded by $O(N!)$ |

## 10.2 ML methods

| Method | Task | Core idea | Important consideration |
|---|---|---|---|
| Linear regression | Numeric prediction | Fit a linear relationship | May miss nonlinear patterns |
| Random forest regressor | Numeric prediction | Average many decision trees | More flexible; not always superior |
| KNN classifier | Classification | Vote among nearby samples | Scaling and $K$ matter |
| SVM classifier | Classification | Learn a margin-based boundary | Scaling, kernel, and $C$ matter |
| ANN classifier | Classification | Learn layered nonlinear transformations | Scaling, training settings, and overfitting matter |

## 10.3 Functions that are easy to confuse

- `fit(X_train, y_train)` trains or fits a model.
- `predict(X_test)` returns predicted labels or numeric predictions.
- `predict_proba(X)` returns class probabilities for estimators that support it.

- `fit_transform(X_train)` learns preprocessing parameters and applies the transformation.
- `transform(X_test)` applies the already learned transformation to test data.
- `compile()` configures a Keras model; `fit()` trains it.
- `axis=1` in pandas `drop` means columns; `axis=0` means rows.

# 11 Common Errors, Debugging, and Responsible Reporting

## 11.1 Dataset and column errors

If Python reports `FileNotFoundError`, verify the current working directory and exact CSV filename. If a `KeyError` occurs, print `df.columns.tolist()` and use the exact spelling, capitalization, and spaces. Different Kaggle downloads can have different names or include an extra unnamed index column. Do not blindly assume the dataset variant matches a tutorial.

## 11.2 Preprocessing and leakage

Do not scale the entire dataset before splitting. Do not select features, tune hyperparameters, or choose thresholds based on test-set results and then present that same test score as unbiased evaluation. Fit imputers, scalers, and other learned transformations on training data only. For model selection, use a validation set or cross-validation within the training data.

## 11.3 Metric mistakes

- Accuracy is not the same as precision or recall.
- $R^2$ is not a classification accuracy percentage.
- RMSE has the same units as the target; it is not bounded by one.
- A confusion matrix must be interpreted with the class ordering in mind.
- A false negative and false positive are defined relative to the chosen positive class.
- Better training performance does not guarantee better test performance.

## 11.4 Reproducible reporting checklist

Before submitting a practical:

1. Record the dataset filename and target column.
2. State how missing values and outliers were handled.
3. State which columns were features and which were removed.
4. Record the train/test split and random seed.
5. For scaled models, state which scaler was used and confirm it was fitted on training data only.
6. State the model settings and evaluation metrics.
7. Report actual output from your own run; do not copy a score from an unrelated tutorial.

8. Mention one limitation and one possible improvement.

# 12 Theory Answer-Writing Framework and Extended Revision

This section gives a repeatable structure for answering descriptive questions. In an exam, start with a precise definition, explain the core mechanism, write the key formula or recurrence, describe the algorithm step by step, give a small example, state complexity, and finish with strengths and limitations. A technically correct definition alone may not demonstrate that the student understands the method.

## 12.1 How to write a strong DAA long answer

For Fibonacci, define the recurrence and base cases, distinguish recursive and iterative execution, explain repeated subproblems, and compare complexity. For Huffman coding, define lossless variable-length prefix coding, describe the min-heap merge process, show how root-to-leaf paths become codes, and explain weighted code length. For fractional knapsack, formulate the capacity constraint and value-density rule, then justify the greedy choice using an exchange argument. For 0–1 knapsack, define the DP state and transition and explain why the previous row prevents reuse of an item. For N-Queens, state the row/column/diagonal constraints, explain the search tree and pruning rule, and describe undoing a placement.

A complexity answer should state what $n$ represents and what assumptions are being made. For example, saying "knapsack is $O(nW)$" refers to the standard dynamic-programming table, where $n$ is the number of items and $W$ is integer capacity. It is not the complexity of every knapsack algorithm. Similarly, Huffman construction with a min-heap is commonly $O(m \log m)$ for $m$ distinct symbols, while encoding a message of length $L$ also depends on $L$ and the code lengths.

## 12.2 How to write a strong ML long answer

For an ML method, define the task and target first, then describe the model's learning principle, preprocessing requirements, training process, and evaluation method. Distinguish parameters learned from data from hyperparameters chosen by the developer. Explain why the train/test split is necessary and how leakage can inflate results. End by interpreting the metrics in the context of the problem rather than reporting one score without explanation.

For Uber fare prediction, identify the numeric fare as the regression target and compare a linear model with a tree ensemble. Explain that location coordinates may need feature engineering and that invalid fares or coordinates should be investigated rather than mechanically deleted. For spam detection, define spam as the positive class if label 1 means spam, explain how KNN uses neighbors and SVM uses a separating boundary, and report confusion-matrix measures. For churn prediction, define churn label 1, explain the ANN layers and training process, and discuss recall for identifying actual churners alongside precision and overall accuracy.

## 12.3 Metric interpretation: what a score can and cannot say

Accuracy is the proportion of correct classifications, but it can hide minority-class failures. Precision asks how often positive predictions are correct; recall asks how many actual positive

examples are found. F1 is the harmonic mean of precision and recall. RMSE is expressed in the same units as the numeric target and penalizes large errors more strongly than MAE. $R^2$ compares squared prediction error with the error from predicting the target mean; it can be negative on held-out data when the model performs worse than that baseline. None of these measures proves that the model is causally correct or that it will perform identically in future conditions.

## 12.4 Worked metric example

Suppose a classifier evaluates 100 emails, 20 of which are spam. It identifies 15 spam emails correctly, incorrectly labels 5 legitimate emails as spam, and misses 5 spam emails. Then $TP = 15$, $FP = 5$, $FN = 5$, and $TN = 75$. Accuracy is $(TP + TN)/100 = 90\%$. Precision for spam is $15/(15 + 5) = 75\%$, and recall for spam is $15/(15 + 5) = 75\%$. The example shows why a confusion matrix gives more information than accuracy alone: it tells us the kinds of errors made.

## 12.5 Worked knapsack distinction

Consider capacity $W = 8$ and three items: $(w, v) = (4, 40), (5, 50), (3, 27)$. Their value densities are 10, 10, and 9. Fractional knapsack can take the first item and then 4/5 of the second, obtaining value $40 + 40 = 80$. For 0–1 knapsack, only whole items are allowed: selecting items of weights 5 and 3 gives value 77, while selecting weight 4 alone gives 40 and selecting weights 4 and 3 gives 67. The fractional solution's ability to take part of an item is exactly why the greedy method and the DP formulation solve different constraints.

## 12.6 Practical checklist before submitting code

Before running an algorithm program, verify the input domain, base cases, termination condition, and expected output on a tiny example. For backtracking, check that state is undone after a failed branch. For DP, write the state definition and recurrence before coding. For ML, print the dataset shape, inspect missing values and target counts, verify feature and target columns, split before fitting transformations, train only on training data, and evaluate on held-out examples. Read the confusion matrix with the class order in mind and do not invent expected scores.

## 12.7 Dataset assumptions and reproducibility notes

The lab list and tutoring discussion identify the problem families, but CSV filenames and column names can differ among downloaded dataset versions. In the Uber example, inspect whether the target is called `fare_amount` and whether pickup/drop-off fields have the expected names. In the spam example, verify the label column and whether identifier columns should be removed. In the churn example, the common Kaggle file uses `Churn_Modelling.csv` and target `Exited`, but the exact supplied file must be checked. These notes describe common variants, not a guarantee about every local file.

For reproducibility, record the dataset version, preprocessing operations, train/test split settings, model hyperparameters, software versions when relevant, and evaluation metrics. A

fixed random seed makes many randomized operations repeatable under the same environment, but it does not mean all numerical libraries or hardware will always produce bit-identical results. A result is meaningful only when its setup is clear enough to interpret and reproduce.

### 12.8 Additional viva questions for understanding

1. Why does naive Fibonacci recursion repeat work, and how does memoization change the complexity?
2. Why must Huffman codewords be prefix-free for unambiguous decoding?
3. State the condition under which the fractional-knapsack greedy rule is optimal.
4. Why does the 0–1 knapsack DP transition refer to the previous row?
5. What must be undone when a backtracking branch fails?
6. Why can accuracy be misleading when churn is rare?
7. Why is scaling especially important for KNN and often useful for SVM and ANN?
8. What is the difference between a model parameter and a hyperparameter?
9. Why should a scaler be fitted only on training data?
10. What do precision and recall mean when churn is class 1?
11. Why is sigmoid suitable for a single binary-classification output?
12. What is the difference between training loss and test performance?

## 13 Final Viva Practice: Questions with Short Answers

### 13.1 DAA revision

1. **What is the difference between recursion and iteration?** Recursion uses function calls on smaller instances; iteration repeats a block using loops.
2. **Why does naive recursive Fibonacci repeat work?** The same Fibonacci subproblems occur in multiple branches.
3. **What makes Huffman code decodable?** Its prefix-free property.
4. **What data structure is useful for Huffman construction?** A min-heap / priority queue.
5. **What is the greedy rule for fractional knapsack?** Descending value-to-weight ratio.
6. **Why does that rule not generally solve 0–1 knapsack?** Items cannot be split, so local ratio choices can block a better combination.
7. **What does $DP[i][c]$ mean?** The maximum value using the first $i$ items with capacity $c$.

8. **What is backtracking?** Explore a choice, recurse, and undo the choice when returning.
9. **How are N-Queens diagonals identified?** Equal $r - c$ or equal $r + c$ values.

## 13.2 Machine-learning revision

1. **What is the difference between regression and classification?** Regression predicts numeric quantities; classification predicts categories.
2. **Why split the data?** To estimate performance on unseen examples.
3. **What is data leakage?** Using information during training or model selection that would not legitimately be available at prediction time.
4. **Why standardize features?** To put numerical features on comparable scales, especially for distance- and margin-based methods.
5. **Why fit a scaler on training data only?** To avoid using test-set statistics in model development.
6. **What is KNN's $K$?** The number of nearest training samples considered for voting.
7. **What is the SVM margin?** The separation region around the decision boundary that the model seeks to maximize.
8. **What is a false negative in churn prediction?** A customer who actually churns but is predicted to stay.
9. **What is the difference between precision and recall?** Precision asks how trustworthy positive predictions are; recall asks how many actual positives were found.
10. **What is an ANN epoch?** One pass through the training data.
11. **What is the role of binary cross-entropy?** It measures the mismatch between binary labels and predicted probabilities.
12. **Does a higher-complexity model always perform better?** No; generalization depends on the data, assumptions, tuning, and overfitting.

## 13.3 Mini oral-exam routine

For each experiment, practise explaining the answer in this order:

1. State the problem and the output expected.
2. Explain the central idea in two or three sentences.
3. Identify the data structures, model, or key library used.
4. Walk through the main code blocks in order.
5. Give the time/space complexity or evaluation metric.
6. Explain one limitation and one improvement.

If you can explain why each line exists, you are better prepared than if you only memorize the program text.

# 14 Machine Learning Experiment 4: Diabetes Prediction Using K-Nearest Neighbors

## 14.1 Experiment statement and objectives

The experiment asks us to implement the K-Nearest Neighbors (KNN) algorithm on the supplied `diabetes.csv` dataset and compute the confusion matrix, accuracy, error rate, precision, and recall. This is a supervised binary-classification task: each row describes a person using clinical or demographic measurements, and the target indicates whether the person belongs to the positive diabetes class. The exact column names and target encoding must be checked in the provided CSV before coding; datasets with similar names do not always have identical schemas.

The objectives are to understand distance-based classification, prepare numerical features consistently, separate training from testing, make predictions, and interpret classification metrics. This experiment is especially useful because it shows why a single accuracy number is not enough when the two classes have different costs or frequencies.

## 14.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Load diabetes.csv -> inspect missing values -> separate X and y -> split
    train/test (stratify) -> fit scaler on training data -> transform
    train/test -> train KNN -> predict -> confusion matrix and metrics
```

## 14.3 What KNN does

KNN is a non-parametric, instance-based learning algorithm. It does not normally learn a compact equation during training. Instead, it stores the training examples. To classify a new example, it computes its distance from training examples, chooses the $K$ closest examples, and assigns the class receiving the most votes. With distance weighting, closer neighbours can receive greater voting weight.

For two observations $x = (x_1, \ldots, x_p)$ and $z = (z_1, \ldots, z_p)$, Euclidean distance is

$$d(x, z) = \sqrt{\sum_j = 1^p (x_j - z_j)^2}.$$

Manhattan distance is $d(x, z) = \sum_j |x_j - z_j|$. Euclidean distance is common for continuous measurements, but the appropriate metric depends on the meaning and scale of the features.

## 14.4 Why feature scaling is essential

Suppose one feature ranges from 0 to 10 while another ranges from 0 to 100,000. Without scaling, differences in the larger-range feature can dominate the distance calculation even if

the smaller-range feature is informative. Standardization transforms each feature using the training-set mean and standard deviation:

$$z = \frac{x - \mu_\text{train}}{\sigma_\text{train}}.$$

The scaler must be fitted only on training data and then applied unchanged to the test data. Fitting the scaler before the split leaks information about the test distribution into the training process. If a feature has zero variance, a robust implementation must handle it safely.

## 14.5 Recommended experimental workflow

1. Read the CSV and inspect its dimensions, column names, data types, missing values, duplicate rows, and target counts.
2. Separate the target column from the predictor columns. Do not accidentally include the label as an input feature.
3. Investigate missing and implausible values. In some diabetes datasets, zero values in measurements such as glucose or blood pressure may be coded as missing; verify the dataset documentation before treating zeros as invalid.
4. Split the observations into training and test sets. Use stratification when appropriate so both sets have similar class proportions.
5. Fit the scaler on the training predictors only; transform both training and test predictors.
6. Train KNN for a chosen value of $K$, predict the held-out test labels, and calculate the requested metrics.
7. Try a small range of $K$ values using validation or cross-validation. Choose the value based on the task's evaluation goal, not by repeatedly tuning on the final test set.

## 14.6 Illustrative Python implementation

The code below is a template. Change `target_column` to the actual target column in the supplied file. Do not copy an assumed column name without inspecting the dataset.

```
import pandas as pd
from sklearn.model\_selection import train\_test\_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion\_matrix, accuracy\_score,
                             precision\_score, recall\_score,
                             classification\_report)

df = pd.read\_csv("diabetes.csv")
print(df.shape)
```

```
print(df.head())
print(df.isnull().sum())
print(df.columns.tolist())
print(df.iloc[:, -1].value\_counts(dropna=False))

# Replace with the verified target column name.
target\_column = "Outcome"
X = df.drop(columns=[target\_column])
y = df[target\_column]

X\_train, X\_test, y\_train, y\_test = train\_test\_split(
    X, y, test\_size=0.20, random\_state=42, stratify=y
)

model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n\_neighbors=5))
])
model.fit(X\_train, y\_train)
y\_pred = model.predict(X\_test)

cm = confusion\_matrix(y\_test, y\_pred)
print("Confusion matrix:\n", cm)
print("Accuracy:", accuracy\_score(y\_test, y\_pred))
print("Error rate:", 1 - accuracy\_score(y\_test, y\_pred))
print("Precision:", precision\_score(y\_test, y\_pred, zero\_division=0))
print("Recall:", recall\_score(y\_test, y\_pred, zero\_division=0))
print(classification\_report(y\_test, y\_pred, zero\_division=0))
```

A pipeline ensures the imputer and scaler are fitted only using the training partition when `fit` is called. If the dataset has categorical features, they need appropriate encoding; this simple pipeline assumes the predictors are numerical. The positive label used by precision and recall should be checked explicitly. In the common Pima-style dataset, `Outcome=1` is usually the positive class, but the supplied file is the authority.

## 14.7 Understanding the confusion matrix

For binary classification, the confusion matrix can be described using four outcomes:

- True positive (TP): a person in the positive class is correctly predicted positive.
- True negative (TN): a person in the negative class is correctly predicted negative.
- False positive (FP): a negative-class person is incorrectly predicted positive.
- False negative (FN): a positive-class person is incorrectly predicted negative.

With the conventional layout from scikit-learn for labels ordered as 0 then 1, the matrix is $\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$. Always confirm the label order when reading a confusion matrix.

## 14.8 Metrics and their interpretation

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Error rate} = \frac{FP + FN}{TP + TN + FP + FN} = 1 - \text{Accuracy}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}.$$

Accuracy is the proportion of all predictions that are correct. Error rate is the proportion that are wrong. Precision asks, "Among people predicted positive, how many were truly positive?" Recall asks, "Among all truly positive people, how many did the model identify?" If missing a possible case is especially costly, recall deserves particular attention, while false positives also have consequences and should not be ignored. A model output is not a medical diagnosis.

## 14.9 Choosing $K$, advantages, and limitations

A very small $K$ can be sensitive to noise and individual examples. A very large $K$ smooths the decision boundary and may overlook local patterns. Odd $K$ values can reduce ties in binary classification, but do not guarantee better results. Cross-validation can compare values such as 3, 5, 7, 9, and 11. Other choices include distance metric, uniform versus distance weighting, and handling of imbalanced classes.

Advantages: simple to understand, little explicit training, and flexible decision boundaries. Limitations: prediction can be expensive because distances to many training rows may be needed; feature scaling matters; irrelevant features can distort distance; high-dimensional data can make neighbourhoods less meaningful; and imbalanced labels can bias voting.

## 14.10 Common viva questions

1. Why is KNN called a lazy learner? It defers most computation until prediction time.
2. Why scale features? Distance is sensitive to the units and ranges of features.
3. What happens if $K = 1$? The prediction depends on the single nearest training observation and may be noise-sensitive.
4. What is the error rate? One minus accuracy, when both use the same test predictions.
5. Why might accuracy mislead? A majority-class classifier can appear accurate while failing to detect many positive cases.
6. Does KNN produce a learned coefficient equation? No; it relies on stored examples and a neighbourhood rule.
7. **What is the target in the common diabetes dataset?** The `Outcome` column, typically coded 0 or 1.
8. **What is X?** The input medical features.
9. **What is y?** The target labels.

10. **Why use `stratify=y`?** To preserve approximate class proportions in train and test sets.
11. **Why set `random_state`?** To make the split reproducible.
12. **What is Euclidean distance?** The square root of the sum of squared coordinate differences.
13. **What does `n_neighbors=5` mean?** The five nearest training examples vote on a classification.
14. **What if K is too small?** Predictions may be sensitive to noise and overfit local patterns.
15. **What if K is too large?** Local class patterns can be smoothed away and the majority class may dominate.
16. **Why fit the scaler only on training data?** To prevent test-set information leaking into preprocessing.
17. **What does `transform()` do?** Applies already learned scaling parameters to new data.
18. **What is accuracy?** The proportion of all predictions that are correct.
19. **What is precision?** True positives divided by predicted positives.
20. **What is recall?** True positives divided by actual positives.

# 15 Machine Learning Experiment 5: Customer Segmentation Using Clustering

## 15.1 Experiment statement and objectives

The prescribed task is to implement K-Means clustering or hierarchical clustering on `sales_data_sample.csv` and determine the number of clusters using the elbow method. This is an unsupervised-learning problem: the dataset does not provide a known class label that the algorithm should predict. Instead, the method groups observations based on similarity in selected features. The exact fields in the supplied sales CSV should be inspected before selecting features.

## 15.2 What is asked and simple coding flow

**What is asked:** Implement the experiment described in the practical statement, produce the required output, and explain the algorithm and its key decisions.

```
Load sales_data_sample.csv -> inspect columns -> choose meaningful numeric
  features -> handle missing values -> scale features -> fit K-Means for
  K=1..10 -> plot inertia (Elbow) -> choose K -> assign cluster labels ->
                            interpret clusters
```

## 15.3 Supervised learning versus clustering

In supervised learning, a target column provides the expected answer during training. In clustering, there is no target label; the algorithm attempts to reveal structure in the observations. Cluster IDs such as 0, 1, and 2 are arbitrary identifiers, not ordered scores and not automatically meaningful business categories. Interpretation comes after clustering by examining the characteristics of each group.

## 15.4 K-Means algorithm

K-Means partitions $n$ observations into $K$ clusters. Each observation is assigned to the cluster with the nearest centroid. A centroid is the mean feature vector of observations currently assigned to that cluster. K-Means attempts to minimize the within-cluster sum of squared distances:

$$J = \sum _i = 1^n \left\| x_i - \mu_c_i \right\|^2 ,$$

where $x_i$ is observation $i$, $c_i$ is its assigned cluster, and $\mu_c_i$ is that cluster's centroid.

The basic procedure is: (1) initialize $K$ centroids, (2) assign each observation to its nearest centroid, (3) recompute each centroid as the mean of its assigned observations, and (4) repeat assignment and update until assignments or centroid positions stop changing significantly or a stopping limit is reached. Different initializations can produce different solutions; K-Means++ initialization and multiple restarts help reduce this problem.

## 15.5 Feature selection and preparation

Read the CSV and inspect its columns, types, missing values, and summary statistics. Remove identifiers that merely label rows, such as an order number, unless the identifier has a justified numeric meaning. Dates and text fields need deliberate treatment. For sales segmentation, candidate numeric features might include quantities, prices, sales totals, or customer/order aggregates, depending on the schema and objective. Do not simply include every numeric column: a code or identifier can create meaningless distances.

Scaling is generally important because K-Means uses distances. A large-range variable can dominate the objective. StandardScaler is a common choice for roughly continuous numeric features; robust scaling may help when strong outliers exist. Scaling changes the meaning of distances, so the selected features and scaling method should be explained in the report.

## 15.6 The elbow method

Run K-Means for a sequence of candidate cluster counts, for example $K = 1$ through $K = 10$, and record the inertia (within-cluster sum of squares) for each fit. Inertia almost always decreases as $K$ increases because more centroids allow points to be closer to a centre. Plot $K$ on the horizontal axis and inertia on the vertical axis. The elbow is the point where the decrease begins to flatten noticeably. It is a heuristic rather than a proof of the true number of clusters; if there is no clear elbow, combine the plot with silhouette score, stability, domain knowledge, and the usefulness of the resulting segments.

## 15.7 Illustrative K-Means implementation

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

sales = pd.read\_csv("sales\_data\_sample.csv")
print(sales.shape)
print(sales.head())
print(sales.dtypes)
print(sales.isnull().sum())

# Select meaningful numeric features after inspecting the CSV.
features = sales.select\_dtypes(include="number").copy()
# Remove numeric identifiers if present and not meaningful.
# Example: features = features.drop(columns=["ORDERNUMBER"])

imputer = SimpleImputer(strategy="median")
X\_imputed = imputer.fit\_transform(features)
scaler = StandardScaler()
X\_scaled = scaler.fit\_transform(X\_imputed)

inertias = []
ks = range(1, 11)
for k in ks:
    km = KMeans(n\_clusters=k, random\_state=42, n\_init=10)
    km.fit(X\_scaled)
    inertias.append(km.inertia\_)

plt.plot(list(ks), inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia / within-cluster sum of squares")
plt.title("Elbow Method")
plt.xticks(list(ks))
plt.show()

# After inspecting the elbow plot, choose K deliberately.
chosen\_k = 3  # illustrative only; use the elbow in your run
final\_model = KMeans(n\_clusters=chosen\_k, random\_state=42, n\_init=10)
labels = final\_model.fit\_predict(X\_scaled)
sales["Cluster"] = labels
print(sales["Cluster"].value\_counts().sort\_index())
print(sales.groupby("Cluster")[features.columns].mean())
```

The value `chosen_k=3` is only an example, not a claim about the supplied dataset. Replace it with a reasoned choice from the actual elbow plot. Feature selection should be refined rather than blindly using all numeric columns. For a rigorous workflow, fit the imputer and scaler consistently and document exactly which columns were used.

## 15.8 How to interpret clusters

After assigning labels, compute per-cluster counts, means or medians, and useful summaries. For example, one cluster might show higher sales values and another lower quantities, but such conclusions must be based on actual output. Name clusters only after inspecting their profiles. Cluster labels can be permuted between runs, so "cluster 0" does not intrinsically mean "best customers." A two-dimensional scatter plot using selected features or PCA can help visualization, but a plot projection can hide differences in the full feature space.

## 15.9 Hierarchical clustering alternative

Hierarchical clustering builds a nested grouping structure. In agglomerative clustering, each observation begins in its own cluster, and the algorithm repeatedly merges the closest clusters until a stopping condition is met. A dendrogram visualizes the sequence and distance of merges. Linkage methods include single, complete, average, and Ward linkage. They define cluster-to-cluster distance differently. Ward linkage commonly works with Euclidean geometry and aims to minimize increases in within-cluster variance. The number of clusters can be selected by cutting the dendrogram at a chosen height, but that decision still requires interpretation.

K-Means is generally efficient for larger numerical datasets but favours roughly compact, centroid-based groups and is sensitive to outliers and initialization. Hierarchical clustering provides a useful dendrogram and does not require the same iterative centroid update, but can be expensive for large datasets. Neither method guarantees that natural, useful groups exist in every dataset.

## 15.10 Common viva questions

1. What is unsupervised learning? Learning patterns from data without a supplied target label.
2. What is a centroid? The mean feature vector of points assigned to a K-Means cluster.
3. What does inertia measure? Sum of squared distances from each point to its assigned centroid.
4. Why does inertia fall when $K$ rises? More clusters give points more nearby centroids; this alone does not prove a larger $K$ is better.
5. What is the elbow method? A visual heuristic that chooses a point where inertia's improvement begins to flatten.
6. Why scale the data? To prevent features with large numeric ranges from dominating distance.
7. Are cluster numbers meaningful? No; labels are arbitrary and require post-clustering interpretation.
8. What is a dendrogram? A tree diagram showing hierarchical cluster merges and their distances.
9. **What does K-Means minimize?** Within-cluster sum of squared distances to centroids.

10. **What is a centroid?** The mean feature vector of the points assigned to a cluster.
11. **Why is clustering unsupervised?** No target labels are required to fit the groups.
12. **What is the x-axis of the elbow plot?** The candidate number of clusters, K.
13. **What is the y-axis?** Inertia, the within-cluster sum of squares.
14. **Does lower inertia alone mean the clustering is better?** No; inertia generally decreases as K grows, so complexity and interpretability matter.
15. **What is a good elbow point?** A candidate K where the reduction in inertia begins to flatten.
16. **Why inspect CSV columns before clustering?** To select meaningful features and avoid identifiers or irrelevant numeric codes.
17. **Why impute missing values?** Most clustering estimators cannot directly process missing numeric values.
18. **Why scale features?** To reduce dominance from features with large numerical ranges.
19. **What does `fit_predict()` return?** A cluster label for every input row.
20. **Are cluster IDs ordered?** No; IDs are arbitrary labels.

# 16 Final Exam Checklist

Before the practical, verify the dataset filename and column names, install the required libraries, run the complete program once, and be able to explain each preprocessing step, algorithm decision, output, and complexity/metric. For the ML clustering experiment, select the final value of $K$ from the actual Elbow curve rather than assuming it is always 3.